NOMAD - Navigating Optimal Model Application to Datastreams

Ashwin Gerard Colaco
University of California, Irvine

Sharad Mehrotra
University of California, Irvine

Michael J De Lucia
USARMY DEVCOM ARL

Kevin Hamlen
The University of Texas at Dallas

Murat Kantarcioglu
Virginia Tech

Latifur Khan
The University of Texas at Dallas

Ananthram Swami
USARMY DEVCOM ARL

Bhavani Thuraisingham
The University of Texas at Dallas

ABSTRACT

NOMAD (Navigating Optimal Model Application for Data-streams) is an intelligent framework for data enrichment during ingestion that optimizes real-time multiclass classification by dynamically constructing model chains—sequences of machine learning models with varying cost-quality tradeoffs, selected via a utility-based criterion. Inspired by predicate-ordering techniques from database query processing, NOMAD leverages cheaper models as initial filters, proceeding to more expensive models only when necessary, while guaranteeing classification quality remains ϵ-comparable to a designated role model through a formal chain safety mechanism. It employs a dynamic belief update strategy to adapt model selection based on per-event predictions and shifting data distributions, and extends to scenarios with dependent models such as early-exit DNNs and stacking ensembles. Evaluation across multiple datasets demonstrates that NOMAD achieves significant computational savings (speedups of 2–6×) compared to static and naive approaches while maintaining classification quality comparable to that achieved by the most accurate (and often the most expensive) model.

PVLDB Reference Format:
Ashwin Gerard Colaco, Sharad Mehrotra, Michael J De Lucia, Kevin Hamlen, Murat Kantarcioglu, Latifur Khan, Ananthram Swami, and Bhavani Thuraisingham. NOMAD - Navigating Optimal Model Application to Datastreams. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Modern data processing systems increasingly require data ingestion components capable of interpreting raw data in real-time. Data ingestion—the process of continuously collecting, importing, and processing data as it arrives—often becomes a critical performance bottleneck [11, 33]. This process involves extracting features, applying trained models, and executing actions based on model outcomes, all while data streams into the system at high rates.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Applications of such real-time ingestion are widespread. For instance, consider a cybersecurity use case: the input data consists of network traffic arriving as a stream, and the goal is to classify each flow into categories such as benign activity or different types of attacks—for example, Distributed Denial of Service (DDoS), port hopping, or more sophisticated intrusion techniques [31]. Such classification may invoke appropriate actions, including triggering countermeasures, tagging the information for database storage, or escalating alerts to cybersecurity analysts. Another example involves dynamic video data captured from live cameras. Ingesting such data may involve running models to classify the video and tag segments based on a given event/entity taxonomy (e.g., start/end of a meeting, appearance of a person or a vehicle, etc.). Object detection models such as YOLO [25], or ResNet [13] can be used to detect and classify objects in footage, while embedding models like CLIP [23] can enable zero-shot classification by comparing image and text representations. Such models can detect attributes like room occupancy or tag segments with specific labels. Such tags facilitate information triage, user-specific distribution, and enable advanced search and analysis capabilities.

These ingestion tasks often correspond to multiclass classification. In the examples above, models like Decision Tree classifiers, Support Vector Machines (SVM), XGBoost, Random Forest, deep neural networks (DNN), CLIP, YOLO, and ResNet can be trained or utilized to differentiate between multiple classes. Such models often exhibit a cost-quality tradeoff. For instance, decision trees are computationally efficient but may not offer the highest accuracy. In contrast, random forests generally perform well for complex tasks but have significantly higher computational costs. Similarly, neural networks can achieve high accuracy, but their performance depends heavily on factors like architecture, size, and data distribution.

A naive strategy is to always run the most accurate model to ensure high-quality results. However, such an approach makes ingestion unnecessarily expensive when cheaper models could produce identical classifications for much of the input data. The challenge lies in achieving quality comparable to the best model while substantially reducing cost.

This paper presents NOMAD, an intelligent ingestion architecture designed to enable efficient, high-quality multiclass classification to address this challenge. NOMAD draws inspiration from cascade classifiers [32] which sequences increasingly complex classifiers to quickly reject easy negative cases while reserving expensive computation for hard examples. Transitioning from binary detection in [32] to multiclass setting introduces fundamental complexity: the choice is no longer whether to stop or to invoke another

(likely more accurate but more expensive) model based on the confidence of the previous model, but also which model to invoke further based on the predictions of prior models. While prior work has explored multiclass cascades (e.g., cost-sensitive tree-structured cascades [36], sequential classification under budget constraints [28], and multiclass boosting cascades [26]), existing work exhibit significant shortcomings when used in dynamic ingestion settings. Existing work has focused on learning a fixed policy of invoking different models at training time. Static policies cannot adapt to distribution shifts common in ingestion streams (evolving attacks, seasonal patterns), nor can they adapt to addition/removal of new models. Furthermore, much of existing work impose model-specific constraints (e.g., boosting cascades [26] require weak learners; tree cascades [36] learn fixed topologies), preventing easy incorporation of heterogeneous pretrained models. Finally, existing approaches do not provide any quality guarantees relative to a designated high-quality model – essential for downstream processing that depends on reliable classifications.

NOMAD addresses these limitations by: (1) providing provable guarantee that the quality of classification achieved will be close to that achievable by the best model in the cascade, (2) dynamically adapting model selections using lightweight statistical techniques (ARIMA, Page-Hinkley) that adjust to distribution shifts without expensive retraining; (3) supporting a model-agnostic design accepting heterogeneous pretrained models as black boxes; and (4) supporting a utility-based selection with dynamic belief updates that navigates the space of model sequences by leveraging per-class accuracy patterns rather than learning fixed cascade structures.

The core idea in NOMAD involves composing models into optimized sequences or decision graphs (model chains) based on expected class distributions and class-specific criteria. NOMAD leverages each model's class-specific performance to determine whether to accept its prediction or proceed to another model in the chain. As the system processes data, NOMAD dynamically selects which models to chain based on ongoing predictions and expected distributions. The framework accommodates both atomic ML models (such as Random Forest, XGBoost, CART, and LDA) and more complex compositions of atomic models. Such complex models may require (possibly multiple) submodels to execute prior to their execution. Examples of such model compositions include ensemble stacking methods that may require component models to be executed before execution of the stacker, and early-exit deep neural networks where invoking the classification at one layer implicitly requires executing prior layers in the network. For ease of exposition, we initially develop NOMAD assuming models operate independently and are invoked on a single event at a time. We then extend NOMAD to scenarios when models used may be dependent where execution follows prerequisite relationships (e.g., early exit neural nets, ensemble methods), and, furthermore, when model invocation is batched amortizing the time overhead of model reloading. We evaluate NOMAD on multiple datasets, demonstrating significant cost savings without compromising classification quality compared to static and naive approaches.

Our contributions include: (1) formalization of the Model Selection Problem with quality guarantees; (2) *chain safety*, characterizing when model chains meet quality guarantees; (3) utility-based model selection with dynamic belief updates to optimize cost while ensuring quality guarantees; (4) extensions of NOMAD; (5) lightweight adaptation using ARIMA and Page-Hinkley drift detection; and (6) experimental validation on eight datasets demonstrating 2–6× speedups while maintaining quality guarantees.

Symbol	Description
E	Stream of events
$C = \{C_1, \ldots, C_k\}$	Set of k classes
$\mathcal{M} = \{M_1, \ldots, M_n\}$	Set of n models
M_r	Role model (quality benchmark)
$\text{cost}(M_i)$	Inference cost of model M_i
D_{val}	Validation dataset
$CM^{(i)}$	Confusion matrix for model M_i
$\text{Q}(M, C_j)$	Quality of model M on class C_j
ϵ	Quality tolerance factor
$\text{EC}(M_i)$	Exit classes for model M_i
$\mathcal{S}$	Classification strategy
S_e	Realized model chain for event e
$Prob(C_j)$	Probability of class C_j

Table 1: Key notation used in this paper.

Section 2 defines the model selection problem. Sections 3 and 4 present NOMAD and its guarantees. Sections 5 and 6 discuss extensions to dependent models & batched execution, and distribution shifts. Section 7 presents experimental results, Section 8 covers related work, and Section 9 concludes.

2 PROBLEM FORMALIZATION

Consider a data stream of events $E = \{e_1, e_2, e_3, \ldots\}$ that are continuously ingested. The objective is to classify each incoming event $e \in E$ into one of several predefined classes $C = \{C_1, C_2, \ldots, C_k\}$ as quickly and accurately as possible. This real-time classification task must balance computational cost and classification quality.

To achieve this, a set of models $\mathcal{M} = \{M_1, M_2, \ldots, M_n\}$ is available. Each model M_i has an associated classification cost, $\text{cost}(M_i)$, and a measure of its classification quality. Without loss of generality, we assume the models are indexed according to their non-decreasing costs: $\text{cost}(M_1) \leq \text{cost}(M_2) \leq \cdots \leq \text{cost}(M_n)$.

In practice, the input workload follows an expected distribution over the classes. We denote the probability of encountering an event of class C_j as $Prob(C_j)$, where $\sum_{j=1}^{k} Prob(C_j) = 1$. This distribution characterizes the typical composition of the event stream.

In the context of the network intrusion detection example from the Introduction, the classes C could represent categories such as normal traffic (C_1), denial-of-service (DoS) attacks (C_2), phishing attempts (C_3), and malware transmissions (C_4).

Table 1 summarizes the key notation used throughout this paper.

2.1 Measure of Model Quality

The performance of each individual model is characterized on a static, representative validation dataset, denoted D_{val}. For a system with k classes, each model M_i produces a $k \times k$ confusion matrix, $CM^{(i)}$, from its predictions on D_{val}. The element $CM^{(i)}_{jl}$ represents the number of events with true class C_j that are predicted as class C_l by model M_i.

$$\begin{bmatrix} 0.900 & 0.050 & 0.030 & 0.020 \\ 0.060 & 0.800 & 0.080 & 0.060 \\ 0.100 & 0.070 & 0.730 & 0.100 \\ 0.030 & 0.040 & 0.050 & 0.880 \end{bmatrix} \quad \begin{bmatrix} 0.950 & 0.030 & 0.010 & 0.010 \\ 0.040 & 0.900 & 0.040 & 0.020 \\ 0.020 & 0.030 & 0.920 & 0.030 \\ 0.015 & 0.025 & 0.030 & 0.930 \end{bmatrix} \quad \begin{bmatrix} 0.980 & 0.010 & 0.005 & 0.005 \\ 0.015 & 0.960 & 0.015 & 0.010 \\ 0.010 & 0.020 & 0.960 & 0.010 \\ 0.005 & 0.015 & 0.010 & 0.970 \end{bmatrix}$$

Table 2: Confusion matrices for classifiers M_1, M_2, and M_3.

Example 2.1. Consider three classifiers evaluated on the same D_{val}. The models may correspond to **Classifier** M_1 (low-cost), **Classifier** M_2 (medium-cost), and **Classifier** M_3 (high-cost). Table 2 shows their confusion matrices, where each entry $CM_{jl}^{(i)}$ represents the probability that model M_i predicts class C_l when the true class is C_j (i.e., $CM_{jl}^{(i)}$ is the count of such events divided by the total number of class C_j events in D_{val}).

From a model's confusion matrix, we can derive various quality metrics such as precision, recall, F1-score, or accuracy. Such quality measures can be defined at the level of individual classes or globally across all classes. We use the notation $Q(M, C)$ for a model M evaluated on class C for class-based metrics, and $Q(M)$ for global metrics which is a macro average over class based quality. Our choice of quality metric is discussed in Section 7.

2.2 Problem Definition

We now formalize the problem of cost-efficient classification.

Definition 2.1 (Role Model). A *role model* $M_r \in \mathcal{M}$ is designated as the benchmark for classification quality. M_r is typically the model with the highest $\mathrm{Q}(M_r, C_j)$ values across all classes, often corresponding to the most expensive model.

Definition 2.2 (Class-based ϵ-Comparability). Given two models $M_i, M_j \in \mathcal{M}$ trained on a given dataset, M_i provides *ϵ-comparable quality* to M_j for a class $C_k \in C$ if its pre-computed quality is within a tolerance factor $(1-\epsilon)$ of M_j's quality:

$$\mathrm{Q}(M_i, C_k) \geq \mathrm{Q}(M_j, C_k) \times (1-\epsilon) \tag{1}$$

Here, $\epsilon \in [0, 1)$ is the *Q tolerance factor* and is the maximum acceptable fractional degradation in quality.

Definition 2.3 (Global ϵ-Comparability). A model M_i provides *global ϵ-comparable quality* to model M_j if its weighted average quality across all classes is comparable:

$$\sum_{k=1}^{|C|} Prob(C_k) \cdot \mathrm{Q}(M_i, C_k) \geq (1-\epsilon) \cdot \sum_{k=1}^{|C|} Prob(C_k) \cdot \mathrm{Q}(M_j, C_k) \tag{2}$$

where $Prob(C_k)$ is the probability of C_k in the expected distribution.

Global ϵ-comparability is weaker than class-based ϵ-comparability - viz., a model satisfying class-based requirement for every class always satisfies the global one, but the converse is not true as we will show in 4.

Definition 2.4 (Exit Model for a Class). Let M_r be the role model and ϵ be the quality tolerance factor. A model $M_i \in \mathcal{M}$ is an *exit model* for class C_j if it provides class-based ϵ-comparable quality to M_r for that class.

Note that according to this definition, multiple models (including M_r itself) can be exit models for the same class C_j.

Strategy A: $CM^{(\mathcal{S}_A)}$

$$\begin{bmatrix} 0.976 & 0.004 & 0.000 & 0.020 \\ 0.100 & 0.810 & 0.002 & 0.088 \\ 0.080 & 0.034 & 0.812 & 0.074 \\ 0.033 & 0.003 & 0.000 & 0.964 \end{bmatrix}$$

Strategy B: $CM^{(\mathcal{S}_B)}$

$$\begin{bmatrix} 0.950 & 0.008 & 0.002 & 0.040 \\ 0.086 & 0.864 & 0.014 & 0.036 \\ 0.078 & 0.016 & 0.870 & 0.036 \\ 0.022 & 0.004 & 0.006 & 0.968 \end{bmatrix}$$

Table 3: Resulting confusion matrices for $\mathcal{S}_A$ and $\mathcal{S}_B$.

Definition 2.5 (Exit Classes for a Model). For a model $M_i \in \mathcal{M}$, its set of *exit classes*, $\mathrm{EC}(M_i)$, are those for which M_i provides class-based ϵ-comparable quality to the role model M_r:

$$\mathrm{EC}(M_i) = \{C_j \in C \mid \mathrm{Q}(M_i, C_j) \geq \mathrm{Q}(M_r, C_j) \times (1-\epsilon)\} \tag{3}$$

By definition, $\mathrm{EC}(M_r) = C$ for any $\epsilon \geq 0$.

Now we define the dynamic components of our system.

Definition 2.6 (Classification Strategy). A *Classification Strategy*, $\mathcal{S}$, defines the dynamic process for selecting and executing one or more models from $\mathcal{M}$ to classify an incoming event.

Definition 2.7 (Realized Model Chain for an Event). For an event e, the *Realized Model Chain*, $S_e = (M^{(1)}, \ldots, M^{(\ell)})$, is the sequence of models executed by strategy $\mathcal{S}$ until a prediction falls into an exit class. Here, $M^{(i)}$ denotes the i-th model executed in the chain for event e, and ℓ is the total number of models executed. Let $\mathrm{Out}(M, e)$ be the predicted class when model M processes event e. The chain terminates at $M^{(\ell)}$ where:

- $\mathrm{Out}(M^{(i)}, e) \notin \mathrm{EC}(M^{(i)})$ for all $i < \ell$.
- $\mathrm{Out}(M^{(\ell)}, e) \in \mathrm{EC}(M^{(\ell)})$.

The final classification for the event is $\mathrm{Out}(\mathcal{S}, e) = \mathrm{Out}(M^{(\ell)}, e)$.

Given a strategy $\mathcal{S}$, we denote its quality as $\mathrm{Q}(\mathcal{S}, C_j)$, which can be computed by simulating the execution of the strategy and evaluating its performance over the validation dataset D_{val}.

Example 2.2. Consider the following two strategies: $\mathcal{S}_A$ and $\mathcal{S}_B$ based on the models in Example 2.1, with M_3 acting as the high-quality role model. **Strategy A**: A full chain $\mathcal{S}_A = (M_1 \rightarrow M_2 \rightarrow M_3)$. **Strategy B**: A shorter chain $\mathcal{S}_B = (M_1 \rightarrow M_3)$. Suppose the exit classes for models M_1 and M_2 are as follows: $\mathrm{EC}(M_1) = \{C_1, C_4\}$ and $\mathrm{EC}(M_2) = \{C_1, C_2, C_4\}$.

To determine quality of a strategy $\mathcal{S}_A$, each event e in D_{val} is first processed by M_1. If M_1 predicts a class in $\mathrm{EC}(M_1)$, the chain terminates with the class predicted by M_1 as the outcome of $\mathcal{S}_A$. Else, e is passed to M_2, and, as before, if the prediction of M_2 is in $\mathrm{EC}(M_2)$, that becomes the prediction of $\mathcal{S}_A$. The process continues until the last model in the chain has executed. By executing $\mathcal{S}_A$ on all events in D_{val} and recording the final predictions versus true labels, we can construct a confusion matrix for $\mathcal{S}_A$. Table 3 shows plausible confusion matrices for the strategies $\mathcal{S}_A$ and $\mathcal{S}_B$ over validation set D_{val}.

Based on these confusion matrices, the quality of the strategy can be determined. For instance, the class-based recall for $\mathcal{S}_A$ and $\mathcal{S}_B$ for the class C_1 are 0.976 and 0.950 and their overall recall is 0.891 and 0.913 respectively.

In addition to quality, we also need to consider the computational cost of a strategy. Since different events may follow different

paths through the model chain (depending on which models' exit conditions are met), the cost varies by event.

Definition 2.8 (Strategy Cost). For a strategy $\mathcal{S}$ and its realized model chain $S = (M^{(1)}, \ldots, M^{(\ell)})$ for a particular event, the **cost of the strategy** is the sum of the costs of all models executed:

$$\text{cost}(S) = \sum_{i=1}^{\ell} \text{cost}(M^{(i)}) \tag{4}$$

where ℓ is the length of the realized chain, and $M^{(i)}$ is the i-th model executed in that chain.

Note that the realized chain, and thus the cost, depends on the specific event being classified. Different events may terminate at different models based on the exit conditions, resulting in variable costs per event across the event stream.

We can now formally define the model selection problem.

Definition 2.9 (Model Selection Problem (MSP)). Given a set of models $\mathcal{M}$, a role model M_r, and a set of classes C, expected distribution $p(C_j)$ for $j = 1, \ldots, |C|$, and tolerance ϵ, the *Model Selection Problem* is to find a **Classification Strategy** $\mathcal{S}$ that minimizes average cost $\mathbb{E}[\text{cost}(S)]$ (expectation over realized chains S for events drawn from the expected distribution) while ensuring the strategy's quality is ϵ-comparable to the role model's quality.

The quality constraint in the model selection problem can be formulated in two ways based on the notion of quality:

(1) **Class-specific Quality Requirement:** For every class, the strategy must maintain quality comparable to the role model:

$$\forall C_j \in C, \quad \text{Q}(\mathcal{S}, C_j) \geq \text{Q}(M_r, C_j) \times (1 - \epsilon)$$

(2) **Global Quality Requirement:** The strategy's weighted average quality must be comparable to the role model:

$$\sum_{j=1}^{|C|} Prob(C_j) \cdot \text{Q}(\mathcal{S}, C_j) \geq (1 - \epsilon) \cdot \sum_{j=1}^{|C|} Prob(C_j) \cdot \text{Q}(M_r, C_j)$$

The class-specific constraint is more stringent, guaranteeing performance for every class including rare ones. The global constraint allows trading off performance across classes, potentially achieving lower costs but risking poor performance on minority classes.

Note that the MSP always has a trivial feasible solution: a strategy that exclusively uses the role model M_r for all events. This satisfies any quality constraint (with $\text{Q}(\mathcal{S}, C_j) = \text{Q}(M_r, C_j)$) but incurs the maximum possible cost. The challenge lies in designing strategies that leverage cheaper models when appropriate while provably maintaining the required quality level.

In the following sections, we present NOMAD, our solution to the Model Selection Problem. NOMAD dynamically constructs model chains that minimize cost while ensuring the quality constraints are satisfied through a combination of strategic model selection, safety checks, and adaptive belief updates.

3 NOMAD ALGORITHM

NOMAD classifies streaming input events while balancing computational cost and classification quality, ensuring final predictions meet a specified quality guarantee relative to role model M_r. It processes each event through an iterative procedure that constructs a sequence of one or more models to derive a classification. Figure 1 depicts this architecture.

The algorithm requires two types of prior information: a probability distribution over classes $C = \{C_1, \ldots, C_k\}$, denoted $Prob(C_j)$ where $\sum_{j=1}^{k} Prob(C_j) = 1$ (representing initial class expectations, assumed known for now; Section 6 discusses learning this distribution), and for each model $M_i \in \mathcal{M}$, pre-computed quality metrics $\text{Q}(M_i, C_j)$, cost, and exit classes $\text{EC}(M_i)$.

Algorithm 1 outlines single-event processing. The process begins with initial beliefs about the event's class (line 1) and enters a loop (lines 2-11) that iteratively selects models, checks safety, and updates beliefs. Each iteration selects the highest-utility model from available models (line 3). If selected and safe (line 5), it executes the model (line 6) and appends to the chain (line 7). If prediction falls into an exit class (line 8), the process terminates; otherwise, beliefs are updated using the model's softmax output (line 10) and the loop continues with remaining models (line 11). If no safe chain exists, the algorithm falls back to M_r (line 12). The key functions 'SelectNextModel' and 'UpdateBeliefs' are detailed below; 'CheckChainSafety', critical for quality guarantees, is detailed in the next section.

3.1 Utility-based Model Selection

The 'SelectNextModel' function selects the model with highest expected utility, defined as the ratio of effectiveness to cost, where effectiveness equals the sum of exit class probabilities based on current belief state p_{current}. The utility score $U(M_i)$ for model M_i given current belief vector p_{current} is:

$$U(M_i) = \frac{\sum_{C_j \in \text{EC}(M_i)} Prob_{\text{current}}(C_j)}{\text{cost}(M_i)}$$

This captures the trade-off between a model's applicability to the current event and its resource consumption. The selection iterates through available models (lines 2-8), computing each model's exit probability (line 3) and utility score (line 4), tracking maximum utility (lines 5-7), and returning the best model (line 9).

3.2 Belief Update

If executed model M^* returns prediction $C_{pred} \notin \text{EC}(M^*)$, the system updates its belief state using 'UpdateBeliefs', which refines the class probability distribution by incorporating the model's softmax output vector $\mathbf{s}$.

The update is a Bayesian operation[18] using element-wise product (Hadamard product) of current probability vector p_{current} and softmax vector $\mathbf{s}$, followed by re-normalization. Non-exit predictions can rule out classes: if a model's classifications for its exit classes are reliable, a non-exit prediction implies the true class is not among those exit classes. The algorithm supports this by zeroing out "ruled-out" classes before the main update. Specifically, the algorithm zeros out probabilities for ruled-out classes (lines 2-3), performs the Hadamard product (line 4), computes the normalization constant (line 5), and normalizes to obtain updated beliefs (lines 6-9). If the normalization constant is too small (indicating numerical instability), it falls back to uniform distribution (line 9). This process is formalized in Algorithm 3.

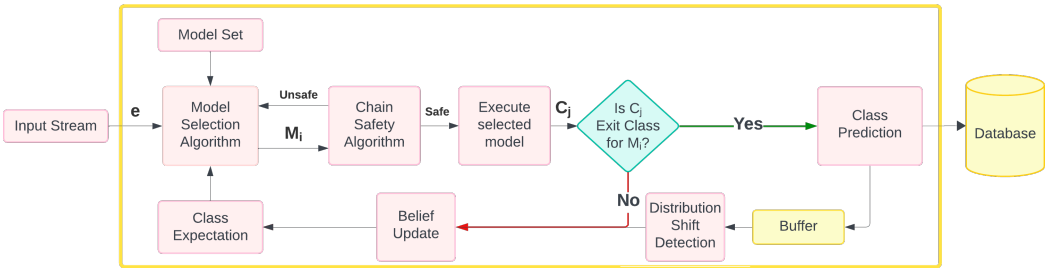

Figure 1: NOMAD's Data Ingestion Framework

Algorithm 1: NOMAD Event Processing

Function *NOMAD_ProcessEvent*$(e, \mathcal{M}, M_r, \mathit{InitialBeliefs}, \epsilon)$
  $\mathcal{M}_{\text{available}} \leftarrow \mathcal{M}$; $Prob_{\text{current}} \leftarrow$ InitialBeliefs;
  $S_{\text{current}} \leftarrow \emptyset$; // The current model chain
  while $\mathcal{M}_{available} \neq \emptyset$ **do**
    $M_{\text{selected}} \leftarrow \text{SelectNextModel}(Prob_{\text{current}}, \mathcal{M}_{\text{available}})$;
    if $M_{selected}$ *is null* **then break**;
    if *CheckChainSafety*$(M_{selected}, S_{current}, M_r, \epsilon, C)$ **then**
      $(C_{\text{pred}}, \mathbf{s}) \leftarrow \text{Execute}(M_{\text{selected}}, e)$;
      $S_{\text{current}} \leftarrow S_{\text{current}} \circ (M_{\text{selected}})$;
      // Append to chain
      if $C_{pred} \in EC(M_{selected})$ **then return** C_{pred};
      // Exit condition met
      else $Prob_{\text{current}} \leftarrow$ UpdateBeliefs$(Prob_{\text{current}}, \mathbf{s}, \text{EC}(M_{\text{selected}}))$;
    $\mathcal{M}_{\text{available}} \leftarrow \mathcal{M}_{\text{available}} \setminus \{M_{\text{selected}}\}$;
  $(C_{\text{final}}, \_) \leftarrow \text{Execute}(M_r, e)$; // Fallback to M_r
  return C_{final};

Algorithm 2: Select Next Model

Function *SelectNextModel*$(p_{current}, \mathcal{M}_{available})$
  $M_{\text{best}} \leftarrow$ null; $U_{\text{max}} \leftarrow -1$;
  foreach *model* $M_i \in \mathcal{M}_{available}$ **do**
    $Prob_{\text{exit\_sum}} \leftarrow \sum_{C_j \in \text{EC}(M_i)} Prob_{\text{current}}(C_j)$;
    $U_i \leftarrow Prob_{\text{exit\_sum}}/\text{cost}(M_i)$;
    if $U_i > U_{max}$ **then**
      $U_{\text{max}} \leftarrow U_i$;
      $M_{\text{best}} \leftarrow M_i$;
  return M_{best};

NOMAD dynamically adapts its classification strategy for each event to minimize computational cost while adhering to quality constraints through utility-based model selection, belief updating, and chain safety (formalized next). The framework generalizes for dependent models (Section 5) and uses distribution shift detection to adapt to long-term data stream changes (Section 6).

Algorithm 3: Update Beliefs

Function *UpdateBeliefs*$(Prob_{in}, \mathbf{s}, C_{ruled\_out})$
  $Prob_{\text{temp}} \leftarrow Prob_{\text{in}}$;
  foreach *class* $C_j \in C_{ruled\_out}$ **do**
    $Prob_{\text{temp}}(C_j) \leftarrow 0$; // Zero out probabilities for ruled-out classes
  $Prob_{\text{new}} \leftarrow Prob_{\text{temp}} \odot \mathbf{s}$;
  // $\odot$ is element-wise product
  $Z \leftarrow \sum_j (Prob_{\text{new}})_j$ // Compute normalization
  if $Z > 10^{-9}$ **then**
    $Prob_{\text{out}} \leftarrow Prob_{\text{new}}/Z$;
  else
    $k \leftarrow$ number of classes in C;
    $Prob_{\text{out}} \leftarrow \{1/k\}_{j=1}^{k}$;
    // Fallback to uniform distribution
  return $Prob_{out}$;

4 CHAIN SAFETY

Chain safety is a fundamental mechanism in NOMAD that ensures quality guarantees are maintained when models are sequenced together. Intuitively, a chain of models is unsafe if it risks failing to achieve ϵ-comparable quality to the role model. This can occur when intermediate models in the chain misclassify events prematurely, preventing those events from reaching a model that would classify them correctly. Chain safety formalizes the conditions under which models can be safely sequenced without this risk.

In this section, we present NOMAD's default chain safety mechanism, which uses a *global* quality guarantee with *relaxed* passthrough estimation. This configuration, used throughout our experiments, provides an effective balance between computational efficiency and quality assurance.

To formalize chain safety, we associate the notions of misclassification probability and passthrough probability with each model. For an event of a given class C_j that is not an exit class for a model M (i.e., $C_j \notin \text{EC}(M)$), the event can be potentially misclassified into one of M's exit classes. We refer to the probability of such an occurrence as the misclassification probability, $\text{Prob}_{mc}(M, C_j)$. This can be estimated from confusion matrix $CM^{(i)}$ for model M_i:

$$\text{Prob}_{mc}(M_i, C_j) = \frac{\sum_{C_k \in \text{EC}(M_i)} CM^{(i)}_{jk}}{N_j}$$

where N_j is the total number of validation instances of class C_j. Note that $\text{Prob}_{mc}(M_i, C_j) \leq 1$ by construction: the numerator $\sum_{C_k \in \text{EC}(M_i)} CM_{jk}^{(i)}$ counts the total number of class C_j events misclassified into exit classes, which cannot exceed the total number of class C_j events, N_j.

Consequently, the passthrough probability, $\text{Prob}_{pt}(M, C_j)$, is the probability that an event of class C_j is not misclassified into an exit class by model M, and is thus allowed to "pass through" to the next model in the chain. It is defined as:

$$\text{Prob}_{pt}(M, C_j) = 1 - \text{Prob}_{mc}(M, C_j)$$

For instance, for our models in Example 2.9, the passthrough probability of a class C_2 event through model M_1 is $\text{Prob}_{pt}(M_1, C_2) = 0.88$, while for a class C_3 event it is $\text{Prob}_{pt}(M_1, C_3) = 0.80$. Likewise, for a C_3 event and model M_2, it is $\text{Prob}_{pt}(M_2, C_3) = 0.92$. Next we show how we compute passthrough probability for a model and then show how we determine if a chain is safe.

4.1 Computing Passthrough Probability

For an event of class C_j that is not an exit class for model M (i.e., $C_j \notin \text{EC}(M)$), the event can potentially be misclassified into one of M's exit classes. We refer to the probability of such an occurrence as the **misclassification probability**, $\text{Prob}_{mc}(M, C_j)$.

Consequently, the **passthrough probability**, $\text{Prob}_{pt}(M, C_j)$, is the probability that an event of class C_j is not misclassified into an exit class by model M, and is thus allowed to "pass through" to the next model in the chain:

$$\text{Prob}_{pt}(M, C_j) = 1 - \text{Prob}_{mc}(M, C_j) \tag{5}$$

We estimate passthrough probabilities using **relaxed estimation with Laplace smoothing**, which uses the entire confusion matrix to provide accurate estimates. For a model M_k predicting class C_p given true class C_t:

$$\text{Prob}_{\text{smoothed}}(C_p|C_t) = \frac{CM_{tp}^{(k)} + \alpha}{N_t + \alpha \cdot |C|} \tag{6}$$

where $\alpha = 1$ (Laplace smoothing). We sum these probabilities over all exit classes to obtain the misclassification probability, and the passthrough estimate is its complement: $\text{Prob}_{pt}^{est}(M_i, C_j) = 1 - \sum_{C_k \in \text{EC}(M_i)} \text{Prob}_{\text{smoothed}}(C_k|C_j)$. With these concepts, we now show how a chain's safety is determined.

4.2 Global Chain Safety

NOMAD supports both global and class-based safety guarantees. We present the global safety mechanism here (our default configuration, see Section 7). The global chain safety mechanism ensures that a strategy's expected quality, weighted by the class probabilities $\text{Prob}(C_j)$, is ϵ-comparable to the role model's quality. This provides a guarantee on average performance across all classes while allowing flexibility for the system to optimize cost by potentially trading off performance across different classes.

For a potential chain $\mathcal{S}$ and a class C_j, the projected quality $Q_{proj}(\mathcal{S}, C_j)$ is the quality of its designated exit model, discounted by the cumulative probability of the event successfully passing through all preceding models. For a chain to be globally safe, the weighted sum of projected qualities must be ϵ-comparable to the role model:

$$\sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q_{proj}(\mathcal{S}, C_j) \geq (1-\epsilon) \cdot \sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(M_r, C_j) \tag{7}$$

Example 4.1. Consider strategies from Example 2.2: $\mathcal{S}_A = (M_1 \rightarrow M_2 \rightarrow M_3)$ and $\mathcal{S}_B = (M_1 \rightarrow M_3)$ with M_3 as role model and $\epsilon = 0.1$. The class distribution is $\text{Prob}(C_1) = 0.4$, $\text{Prob}(C_2) = 0.3$, $\text{Prob}(C_3) = 0.2$, and $\text{Prob}(C_4) = 0.1$.

For $\mathcal{S}_A$, $\text{EC}(M_1) = \{C_1, C_4\}$ and $\text{EC}(M_2) = \{C_1, C_2, C_4\}$. From Table 2's confusion matrices, passthrough probabilities using relaxed estimation are $\text{Prob}_{pt}(M_1, C_2) = 0.91$, $\text{Prob}_{pt}(M_1, C_3) = 0.85$, and $\text{Prob}_{pt}(M_2, C_3) = 0.92$.

Projected quality for each class discounts the exit model's quality by cumulative passthrough probability.

For C_1, which exits at M_1, $Q_{proj}(\mathcal{S}_A, C_1) = Q(M_1, C_1) = 0.90$.

For C_2, which exits at M_2, $Q_{proj}(\mathcal{S}_A, C_2) = \text{Prob}_{pt}(M_1, C_2) \times Q(M_2, C_2) = 0.819$.

For C_3, which exits at M_3, $Q_{proj}(\mathcal{S}_A, C_3) = \text{Prob}_{pt}(M_1, C_3) \times \text{Prob}_{pt}(M_2, C_3) \times Q(M_3, C_3) = 0.751$.

For C_4, which exits at M_1, $Q_{proj}(\mathcal{S}_A, C_4) = Q(M_1, C_4) = 0.88$.

The global projected quality is:

$$\sum_{j=1}^{4} \text{Prob}(C_j) \cdot Q_{proj}(\mathcal{S}_A, C_j) = 0.844$$

The safety threshold is $(1 - \epsilon)\times$ the role model's global quality. With M_3 having per-class qualities 0.98, 0.96, 0.96, and 0.97 for C_1 through C_4:

$$(1 - 0.1) \times [0.4(0.98) + 0.3(0.96) + 0.2(0.96) + 0.1(0.97)] = 0.872$$

Since $0.844 < 0.872$, $\mathcal{S}_A$ is globally unsafe at $\epsilon = 0.1$. The longer chain causes too many C_3 misclassifications, dragging down overall quality.

However, strategy $\mathcal{S}_B = (M_1 \rightarrow M_3)$ requires events to pass through only one model before reaching M_3 for non-exit classes, reducing the cumulative misclassification risk. For C_1 and C_4 that exit at M_1, projected qualities are 0.90 and 0.88. For C_2 and C_3 that exit at M_3, projected qualities are $0.91 \times 0.96 = 0.874$ and $0.85 \times 0.96 = 0.816$ respectively.

The global projected quality for $\mathcal{S}_B$ is:

$$0.4(0.90) + 0.3(0.874) + 0.2(0.816) + 0.1(0.88) = 0.873$$

Since $0.873 > 0.872$, $\mathcal{S}_B$ is globally safe at $\epsilon = 0.1$. By using a shorter chain, NOMAD successfully constructs a strategy that maintains quality while reducing cost. This illustrates how the global safety check identifies viable strategies by balancing quality across the entire class distribution and preferring shorter chains when they suffice.

4.2.1 Checking Global Chain Safety. Algorithm 4 formalizes the global chain safety checking process. The algorithm forms a potential chain by appending the new model (line 1), then computes the global projected quality (lines 2-13) and checks if it meets the safety threshold (lines 14-15).

For each class (line 3), the algorithm computes the cumulative passthrough probability by multiplying the passthrough probabilities of all non-exit models (lines 6-12), identifies the exit model

(lines 7-10), and accumulates the weighted projected quality (line 13). The check on line 15 uses '<' because it specifically tests for the unsafe condition: if the global projected quality is less than the threshold, the chain is unsafe and the function returns false. The chain is considered safe (returns true on line 16) only if the global projected quality exceeds the threshold.

Before a model is added to an event's chain, this function is called to ensure the new, longer chain remains globally safe.

Algorithm 4: Check Global Chain Safety

Input: M_{new}, $\mathcal{S}_{current}$, M_r, ϵ, C
Output: true if chain is safe, false otherwise

1 $\mathcal{S}_{potential} \leftarrow \mathcal{S}_{current} \circ (M_{new})$; // Append new model
2 $Q_{global\_proj} \leftarrow 0$;
3 **foreach** *class* $C_j \in C$ **do**
4 $Prob_{pass\_cum} \leftarrow 1.0$;
5 $M_{exit} \leftarrow$ null;
6 **foreach** *model* $M_k \in \mathcal{S}_{potential}$ **do**
7 **if** $C_j \in EC(M_k)$ **then**
8 $M_{exit} \leftarrow M_k$;
9 **break**;
10 **else**
11 $Prob_{pass\_cum} \leftarrow Prob_{pass\_cum} \times \text{Prob}_{pt}(M_k, C_j)$; // Must pass through
12 **if** $M_{exit} = null$ **then**
13 $M_{exit} \leftarrow M_r$; // Fallback to RM
14 $Q_{global\_proj} \leftarrow Q_{global\_proj} + \text{Prob}(C_j) \times Prob_{pass\_cum} \times Q(M_{exit}, C_j)$;
15 $Q_{threshold} \leftarrow (1-\epsilon) \times \sum_{j=1}^{|C|} \text{Prob}(C_j) \times Q(M_r, C_j)$;
16 **if** $Q_{global\_proj} < Q_{threshold}$ **then**
17 **return** false ; // Chain is unsafe
18 **return** true ; // Chain is globally safe

Theorem 4.1. *A classification strategy $\mathcal{S}$ constructed to satisfy the global chain safety condition meets the Global ϵ-comparability requirement, if the estimated passthrough probabilities, $Prob_{pt}^{est}$, are less than or equal to the true probabilities $Prob_{pt}^{true}$.*

Proof. Let the realized quality of a strategy $\mathcal{S}$ for class C_j be $Q(\mathcal{S}, C_j)$. By definition, this quality is the product of the true cumulative passthrough probability and the quality of the exit model, M_{exit}:

$$Q(\mathcal{S}, C_j) = \left(\prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{true}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The projected quality is calculated using estimated probabilities:

$$Q_{proj}(\mathcal{S}, C_j) = \left(\prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{est}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The theorem's premise is that for any model, $\text{Prob}_{pt}^{true}(M_k, C_j) \geq \text{Prob}_{pt}^{est}(M_k, C_j)$. For relaxed estimation, this premise holds under the standard assumption that the validation set is representative of the true data distribution. Since probabilities are non-negative, the cumulative product also holds this inequality. Therefore, the realized quality is greater than or equal to the projected quality: $Q(\mathcal{S}, C_j) \geq Q_{proj}(\mathcal{S}, C_j)$.

Multiplying both sides by the non-negative class probability $\text{Prob}(C_j)$ and summing over all classes:

$$\sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(\mathcal{S}, C_j) \geq \sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q_{proj}(\mathcal{S}, C_j)$$

The strategy is constructed to satisfy the global safety condition (Equation 7). Combining these inequalities yields:

$$\sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(\mathcal{S}, C_j) \geq (1-\epsilon) \cdot \sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(M_r, C_j)$$

Hence, the realized quality ensures global ϵ-comparability. □

The global chain safety mechanism with relaxed passthrough estimation provides an effective practical approach that balances quality guarantees with computational efficiency. Alternative approaches—including conservative passthrough estimation and class-based safety guarantees—are discussed in the appendix.

5 EXTENSIONS

This section describes two main extensions to the NOMAD framework that have a significant impact on our evaluation. The appendix also presents an alternative formulation of the model selection problem as a Markov Decision Process (MDP), in contrast to the utility-based greedy algorithm described here. The two approaches achieve comparable performance, with the MDP-based method showing only marginal improvements but at the cost of substantially higher complexity—particularly in dynamic settings that require costly retraining. For this reason, we focus in the main body on the greedy algorithm, while the MDP-based formulation and its comparison are discussed in the longer version.

Extension to Dependent Models. NOMAD extends to scenarios where model executions have dependencies, such as early-exit DNNs [27] (where intermediate classifiers depend on preceding layers) and stacking ensembles [8].

We represent inter-model relationships as a DAG $G = (\mathcal{M}, E)$, where edge $(M_i, M_j) \in E$ signifies that M_j requires M_i. The prerequisite set $\text{Prereq}(M_i)$ comprises all models that must execute before M_i. The core NOMAD loop adapts by tracking executed models $\mathcal{M}_{\text{exec}}$ to determine ready models $\mathcal{M}_{\text{ready}}$ at each iteration. A model becomes ready once all prerequisites are in $\mathcal{M}_{\text{exec}}$.

Since dependent models reuse computation from prerequisites, we redefine cost as **incremental cost**:

$$\text{inc\_cost}(M_i, \mathcal{M}_{\text{exec}}) = \text{cost}(M_i) - \sum_{M_j \in \text{Prereq}(M_i) \cap \mathcal{M}_{\text{exec}}} \text{shared\_cost}(M_j, M_i) \quad (8)$$

where $\text{shared\_cost}(M_j, M_i)$ is the reused computation from M_j. The utility score becomes state-dependent:
$U(M_i, \mathcal{M}_{\text{exec}}) = \frac{\sum_{C_j \in \text{EC}(M_i)} p_{\text{current}}(C_j)}{\text{inc\_cost}(M_i, \mathcal{M}_{\text{exec}})}$, ensuring models that leverage prior computation have higher utility.

Batched Inference. Our implementation processes events in **micro-batches** rather than individually. We use a timing-based batching approach where incoming events are accumulated until the batch reaches size N_{batch} or a timeout occurs (50ms). The batch is then processed through model chains with dynamic routing: after each model executes on the batch, events whose predicted class falls into an exit class terminate immediately, while remaining events update their beliefs and continue to the next selected model. This provides early exit benefits at the batch level—events classified by cheaper models terminate immediately, while only difficult events proceed to expensive models.

We use $\mathbf{N_{batch} = 200}$ as our default, which balances throughput and latency. Batching provides substantial speedups: 2-3× for simple models (LDA, CART), 3-5× for tree ensembles (RF, XGB), and 5-10× for deep learning models through better vectorization and hardware utilization. NOMAD's dynamic routing amplifies these benefits—because easier events exit early, expensive models only process the reduced subset of difficult events.

6 ADAPTIVE ALGORITHM

Our description of NOMAD has so far assumed a fixed class distribution. In real-world scenarios, however, this distribution can dynamically shift over time, a phenomenon known as **distributional drift**. To remain responsive to even sudden changes, NOMAD incorporates a mechanism to detect drift and adapt its class priors, $Prob(C)$, on an event-by-event basis. We deliberately choose lightweight statistical techniques for this task. While more complex machine learning models like LSTMs [15] or TimesFM [7] exist for time-series analysis, their computational overhead in an online setting could negate the very efficiency gains NOMAD is designed to provide while providing minimal forecasting accuracy gain. Our approach therefore prioritizes efficiency by using two classical statistical tools in tandem.

The first tool is the **Autoregressive Integrated Moving Average (ARIMA)** model[3], a time-series forecasting technique. We use a set of per-class ARIMA models to predict the probability of the next event belonging to a certain class based on the sequence of past events. The second tool is the **Page-Hinkley (PH) test**[22], a variant of CUSUM, a classic sequential analysis algorithm designed to detect abrupt changes in the average value of a data stream. We use it to monitor the performance of our ARIMA forecasts; a sudden drop in accuracy, flagged by the PH test, signals a potential distribution drift. The complete process is formalized in Algorithm 5.

The adaptive loop begins by using the ARIMA models to forecast the class distribution for the current event (Algorithm 5, Line 6). When the event's true class, C_{obs}, is observed (Line 7), we quantify the model's performance by calculating a "surprise" value: the negative log-likelihood of the observed class(Line 9). This stream of residual values is fed into the Page-Hinkley (PH) test (Line 10). The PH test tracks two running variables initialized to zero: $\boldsymbol{m_t}$, a cumulative sum of the residuals adjusted by their running average, and $\boldsymbol{M_t}$, the minimum value this sum has reached so far. A drift is flagged (Line 11) when the difference $m_t - M_t$ exceeds a predefined threshold λ. Intuitively, this condition means that the model's prediction errors are consistently increasing, which strongly suggests that the underlying class distribution has changed.

Algorithm 5: Event-by-Event Adaptive Priors with ARIMA and Page-Hinkley Test

Input: Stream of new classified events $\mathcal{E}$; PH test parameters: threshold λ, tolerance δ; Retraining buffer size N;

1 **Function** *AdaptivePriorUpdate()*
2 Initialize ARIMA models $\{A_i\}$ with historical data;
3 Initialize PH detector: $ph \leftarrow (m = 0, M = 0)$;
4 Initialize event buffer $\mathcal{B}$ (size N);
5 **for** *each new classified event e in* $\mathcal{E}$ **do**
6 $Prob_{\text{pred}} \leftarrow$ Predict distribution for e using $\{A_i\}$;
7 $C_{\text{obs}} \leftarrow$ Get class of event e;
8 Add C_{obs} to buffer $\mathcal{B}$;
9 $residual \leftarrow -\log(Prob_{\text{pred}}[C_{\text{obs}}])$;
10 Update ph with $residual$ using PH update rule;
11 **if** *ph signals a drift* $(m_t - M_t > \lambda)$ **then**
12 Retrain ARIMA models $\{A_i\}$ using events in $\mathcal{B}$;
13 Reset PH detector $ph \leftarrow (m = 0, M = 0)$;
`// Adaptation triggered`
14 Incrementally update $\{A_i\}$ with C_{obs};
15 $Prob_{\text{current}}(C) \leftarrow$ Get forecast from updated $\{A_i\}$;

This system employs a two-tier adaptation strategy to be both responsive and efficient. The first tier is a low-cost **incremental update** performed after every single event (Line 14). This step does not retrain the model from scratch. Instead, it feeds the latest observed class (C_{obs}) and its corresponding forecast error back into the existing ARIMA model. This updates the model's short-term memory of past observations (for its AutoRegressive component) and past errors (for its Moving Average component). This ensures the forecast for the very next event is always based on the most current information.

The second, more intensive tier is a full **retraining** (Line 12). This is triggered only when the PH test detects a significant drift, suggesting the underlying data patterns have fundamentally changed. In this case, the system re-estimates the core coefficients of the ARIMA models from scratch using the recent history of events stored in a buffer. This dual approach provides the benefits of immediate, event-by-event adjustments while reserving computationally expensive retraining for when it is truly necessary, thus maintaining the overall efficiency of NOMAD.

7 EVALUATION

We evaluate NOMAD across eight diverse datasets spanning tabular, image, and text modalities, comparing its cost-quality performance against baseline approaches. Our evaluation demonstrates NOMAD's ability to achieve significant speedups while maintaining formal quality guarantees, examines its throughput advantages under realistic workloads, and validates its adaptive mechanisms under distribution drift.

7.1 Experimental Setup

Datasets. We selected eight publicly available datasets which include tabular data for network intrusion detection and activity recognition, image data for object recognition, and text data for sentiment analysis. This variety allows us to assess NOMAD's generalizability.

Table 4 summarizes the datasets, listing for each dataset (a) the number of features, (b) number of classes, (c) a metric *PS* that quantifies the maximum achievable cost savings of any algorithm that selects a cheaper model while maintaining ϵ-comparability to the role model, and (d) S_{max}, which is the theoretically maximum speedup that can be achieved such an algorithm on the dataset. The value of *PS* and S_{max} depend upon ϵ and the role model (as will be clear below), and the reported values correspond to $\epsilon = 0.1$ using the stacker as the role model. As ϵ increases (i.e., as more quality degradation is tolerated), more cheap models quality as ϵ-comparable, both *PS* and S_{max} increase accordingly.

PS is computed by, for each validation event with true class c, selecting the cheapest model $M_{c,cheap}$ that is ϵ-comparable to the role model for that class. Such a selection can only be made by an **ideal oracle with perfect knowledge**. The cost saving is:

$$\text{Saving} = \frac{\text{cost}(M_r) - \text{cost}(M_{c,cheap})}{\text{cost}(M_r)}$$

PS is the average of these savings across all validation events, weighted by the class distribution. Since *PS* is calculated based on the ideal oracle, it represents an **upper bound** on the savings attainable by any model selection algorithm. The **theoretical maximum speedup** ($S_{\max}$) for a dataset can, thus, be computed as $S_{\max} = 1/(1 - PS)$. For example, UNSW-NB15 has $PS = 0.74$, meaning that an oracle could save 74% of the cost on average, corresponding to a maximum speedup of $S_{\max} = 1/(1-0.74) = 3.85$. Our goal in defining *PS* and $S_{\max}$ is to determine how close NOMAD selection approach based on utility can approach the upper bound on savings/speedup.

Experimental Hardware. Our evaluation used a server-grade system (Intel Xeon 6706P-B Processor, 256 GB RAM, Ubuntu 22.04 LTS) with 8 CPU workers for parallel execution.

Model Portfolio. We trained a portfolio of 12-15 models per dataset using architectures appropriate for each data modality.

Base Models: For tabular datasets, we used traditional machine learning models including Linear Discriminant Analysis (LDA), Classification and Regression Trees (CART), K-Nearest Neighbors (KNN), Random Forest (RF), XGBoost (XGB), and Multi-layer Perceptrons (MLP) with varying depths. For image data (CIFAR-10), we employed Convolutional Neural Networks (CNNs) ranging from simple 3-layer architectures to deeper VGG-style and ResNet models. For text data (Twitter Sentiment), we used Logistic Regression on TF-IDF features, feedforward networks, and Transformer-based models like DistilBERT.

Ensemble Models: We trained two types of ensemble models: (1) *Stacking Ensembles*: Meta-models that combine predictions from multiple base models using Logistic Regression or XGBoost meta-learners. (2) *Early-Exit DNNs*: Neural networks with intermediate classification layers at different depths, allowing early termination for easy instances.

All models were trained to convergence on a training set and evaluated on a separate validation set to compute confusion matrices and per-class quality metrics. While the portfolio contains 12-15 models, *the number of possible model selection strategies (cascades) grows combinatorially.* NOMAD's utility-based approach efficiently navigates this large space by selecting models dynamically based on class expectations rather than enumerating all possible cascades.

Dataset	Feat.	Cls.	PS	$S_{\max}$
Tabular Datasets				
UNSW-NB15 [1]	49	10	0.74	3.85
CIC-IoT [4]	47	34	0.65	2.86
ACI IoT [2]	44	8	0.78	4.55
CIC-Flow [5]	79	10	0.81	5.26
UCI HAR [30]	561	6	0.86	7.14
ForestCover [29]	54	7	0.85	6.67
Image Dataset				
CIFAR-10 [19]	3072	10	0.65	2.86
Text Dataset				
Twitter Sentiment [10]	Text	3	0.69	3.23

Table 4: Overview of Datasets.

Baselines. We compare NOMAD against two baseline strategies: (1) **Role Model Only**: Always executes the role model for every event in the batch. **Multi-Class Confidence Cascade**: We adapt the confidence-based cascade approach from [28, 35], which processes events sequentially through models ordered by increasing cost, exiting when a model's prediction confidence exceeds a predefined threshold. We select this as our primary baseline because it represents the standard approach for cost-sensitive multiclass classification and shares NOMAD's goal of using cheaper models when possible. Unlike training-time cascade methods [26, 36] that learn fixed cascade structures during training, confidence-based cascades can operate over arbitrary pretrained model portfolios, making them directly comparable to NOMAD. Note that neither training-time cascade methods nor confidence-based cascades provide formal quality guarantees—their confidence thresholds and learned boundaries are heuristic and do not ensure ϵ-comparability to a role model. We implement this baseline using the same model portfolio available to NOMAD and tune the confidence threshold to achieve the best possible cost-quality tradeoff, representing the strongest possible configuration of this approach.

Metrics and Evaluation Protocol. We measure cost as average inference time per event on our evaluation hardware with batched execution. We measure quality using macro-averaged F1-score for overall performance and per-class F1-scores for granular analysis. The stacking ensemble achieved the highest F1-score across all datasets and serves as our default role model (M_r). Our primary efficiency metric is speedup, calculated as the ratio of the Role Model's cost to NOMAD's cost. All results use 5-fold cross-validation with error bars showing ±1 standard deviation.

Default Configuration. Unless stated otherwise, experiments use the following settings: $\epsilon = 0.10$ (quality tolerance), stacker as role model, global safety mode with relaxed passthrough estimation, and batched inference with $N_{\text{batch}} = 200$ as described in Section 5.

With batching, models achieve substantial throughput improvements: simple models (LDA, CART) process events at 0.9-3.3ms each, medium-complexity models (RF, XGB) at 3.9-6.2ms each, deep learning models at 8-15ms each, and ensemble models at 15-25ms each (per-event equivalent within batches).

7.2 Experiments

We begin with an experiment to verify that NOMAD ensures ϵ-comparability and to demonstrate that alternative cascade approaches do not provide such guarantees. We then evaluate NOMAD's cost-quality tradeoff and its impact on ingestion throughput, before examining its robustness under distribution drift.

7.2.1 Quality Preservation. Figure 2 shows that NOMAD successfully maintains the ϵ-comparability guarantee across all datasets while the cascade baseline frequently violates quality constraints.

NOMAD achieves F1-scores between 0.91 and 1.03 times the role model's F1-score (mean 0.96), staying well above the 0.90 threshold required by $\epsilon = 0.10$. Normalized scores above 1.0 indicate that NOMAD outperforms the role model; this occurs on some datasets (e.g., UCI HAR at 1.03) when cheaper models happen to perform better on frequently-occurring classes.

The cascade baseline, despite using the same models, achieves only 0.62-0.94 normalized quality. It violates *the quality constraint on 6 out of 8 datasets because confidence scores do not reliably indicate per-class quality*. High-confidence predictions can still be wrong, especially for minority classes.

This demonstrates NOMAD's key advantage: formal quality guarantees through confusion matrix analysis rather than heuristic confidence thresholds.

7.2.2 NOMAD Speedup. To evaluate NOMAD's performance, we compare the speedup achieved on different datasets against the theoretical maximum speedup, $S_{\max}$, that can be achieved by an oracle at the default setting ($\epsilon = 0.10$).

Figure 2(b) shows NOMAD achieves speedups ranging from 2.1× (CIFAR-10, CIC-IoT) to 6.4× (UCI HAR), capturing 75-90% of the theoretical maximum speedup across all datasets. This demonstrates that NOMAD's utility-based heuristic effectively approximates optimal model selection.

To evaluate how closely NOMAD approaches the theoretical optimum across different quality tolerances (ϵ), Figure 3 shows, for each dataset, the ratio between the speedup achieved by NOMAD and the maximum possible speedup (S_{max}) as ϵ varies from 0.05 to 0.2. As illustrated, this ratio consistently remains between 0.70 and 0.95 across all ϵ values, demonstrating that NOMAD delivers near-optimal performance throughout the tested range.

7.2.3 Ingestion Throughput with NOMAD. We next consider the impact on ingestion throughput under realistic workload conditions. Figure 4(a) demonstrates NOMAD's throughput advantage by simulating an event stream with varying arrival rates for the UNSW-NB15 dataset using 8 parallel workers.

With batched execution, the role model baseline achieves 457 events/second while NOMAD achieves 1494 events/second—a 3.3× improvement that matches the speedup ratio. This improvement comes from two synergistic effects:

(1) Lower per-event cost: NOMAD processes most events with cheap models (5.3ms average) versus the role model's uniform high cost (17.5ms).

(2) Asymmetric batching benefits: NOMAD's early-exit behavior creates an asymmetry that amplifies its advantage. Cheap models process full batches (200 events) for the majority of easy cases, maximizing parallelization efficiency. Expensive models only process residual mini-batches (20-40 events) of hard cases that couldn't exit early. This means cheap models achieve near-optimal batching throughput on most events, while the baseline must always use expensive models with full batches on all events.

As arrival rates increase, both approaches eventually reach their thrashing points where queuing delays dominate. The role model thrashes at 457 events/s while NOMAD sustains up to 1494 events/s, providing substantial headroom for handling traffic spikes or reducing hardware requirements.

7.2.4 Impact of Distribution Drift. Real-world data streams often experience distribution shifts over time. User behavior changes, attack patterns evolve, and seasonal trends emerge. NOMAD's utility-based model selection depends on class probabilities, so these shifts can degrade performance if left unaddressed. We evaluate NOMAD's adaptive mechanism that detects distribution changes and updates its selection strategy.

We simulate an abrupt distribution shift in a stream of 10,000 events. The shift occurs at event 5,000, changing the class distribution significantly. Figure 4(b) shows the average cost per event over time, comparing adaptive and non-adaptive versions of NOMAD.

Before the shift, both versions perform identically at about 5.3ms per event (consistent with UNSW-NB15 batched performance). After the shift, the non-adaptive version immediately jumps to 7.4ms per event and stays there—a 40% cost increase. This happens because NOMAD continues using the old class distribution to make model selection decisions. The utility function prioritizes models that were optimal for the old distribution but are suboptimal for the new one.

The adaptive version also experiences an initial cost spike, but quickly recovers. The Page-Hinkley test detects the shift within about 200 events by monitoring the ARIMA forecast errors. Once detected, NOMAD retrains its distribution model using the 1,000 most recent events. Within 500 events after the shift, the adaptive version returns to optimal performance at 5.3ms per event.

The overhead of adaptation is minimal. Retraining takes only a few milliseconds per shift detection and occurs infrequently. Over the 5,000 post-shift events, the non-adaptive version incurs 10.5 extra seconds of cost (5,000 events × 2.1ms extra per event), while the adaptive version incurs only 2.6 extra seconds during recovery—a 4× reduction in drift-related overhead.

7.2.5 When Does Adaptation Help? Figure 5(a) shows how the benefit of adaptation scales with drift intensity. We measure drift using KL divergence between the old and new distributions. At KL = 0 (no drift), there is no difference between adaptive and non-adaptive versions. As drift increases, adaptation prevents progressively more cost increase.

At moderate drift (KL = 0.4), adaptation prevents 20-35% cost increase across datasets. At severe drift (KL = 0.8), it prevents 30-50% increase. This shows that adaptation provides substantial value

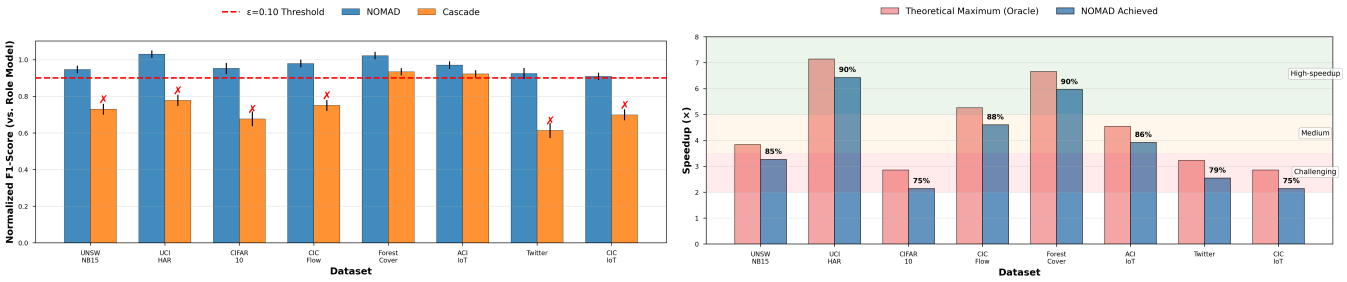

Figure 2: a) Quality comparison at $\epsilon = 0.10$. Error bars show ±1 std dev. b) NOMAD speedup compared to S_{max} across datasets at $\epsilon = 0.10$.

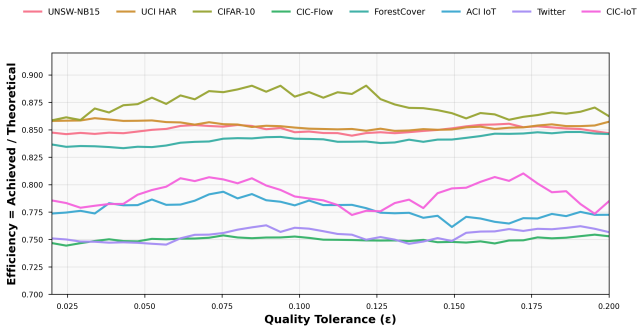

Figure 3: NOMAD Efficiency vs Oracle for varying ϵ values.

in dynamic environments, with benefits growing as conditions become more volatile.

Some datasets benefit more from adaptation than others. CIC-IoT and ACI IoT show large benefits (45-52% at high drift) because their class distributions are imbalanced—shifts dramatically change which models are optimal. UCI HAR and ForestCover show smaller benefits (20-25%) because cheap models perform well across most distributions, limiting the impact of suboptimal selection.

7.2.6 Tuning the Detection Threshold. The Page-Hinkley test uses a threshold λ to control detection sensitivity. Figure 5(b) shows the tradeoff: lower λ detects shifts faster but may trigger false alarms on noisy data, while higher λ is more stable but slower to detect genuine shifts.

Across datasets, detection delay scales roughly linearly with λ. At $\lambda = 100$ (our default), most datasets detect shifts within 140-180 events. At $\lambda = 250$, detection takes 310-450 events. The Twitter dataset is most sensitive to noise and benefits from higher λ, while stable datasets like UCI HAR can use lower λ for faster detection.

In practice, λ should be tuned based on the expected volatility of the data stream. For stable environments, use $\lambda = 50 - 100$ for fast detection. For noisy or highly variable streams, use $\lambda = 150 - 250$ to avoid false alarms. Our experiments use $\lambda = 100$ as a reasonable default that balances detection speed and stability.

The retraining buffer size N (how many recent events to use for retraining) also affects performance. We use $N = 1,000$ by default, which provides enough data to estimate the new distribution accurately without including too much pre-drift data. Analysis of different N values is provided in the appendix.

7.3 Summary of Experimental Evaluation

Our evaluation demonstrates that NOMAD achieves 2-6× speedups (50-84% cost reductions) while maintaining ϵ-comparability guarantees, capturing 75-90% of the theoretical maximum performance across eight diverse datasets spanning tabular, image, and text modalities. NOMAD successfully preserves quality where confidence-based cascades fail, sustains 3-4× higher ingestion throughput than always using the role model, and adapts to distribution shifts to prevent 20-50% cost increases that would otherwise occur.

The extended paper contains additional experiments: comparison of NOMAD's greedy algorithm to the MDP formulation (Sec 5) (both have almost similar performance); experiments to establish NOMAD's robustness to poor initial class priors (NOMAD quickly learns class distribution with minimal impact to performance); NOMAD's performance across different metrics (class-based vs global), as expected, about 15-30% higher speedups are achieved for global metrics; detailed study of the overhead NOMAD incurs in selecting models (it is negligible compared to its savings), and experiments on varying batch size ($N_{\text{batch}} = 200$) is in the optimal range.

8 RELATED WORK

We discuss related work in database query processing, adaptive ML, and scalable data ingestion systems.

Efficient Data Ingestion under Constraints. Techniques like *batching* are commonly used to amortize overheads and improve ingestion throughput [11, 33]. Such techniques are complementary to NOMAD. NOMAD's primary contribution lies in the intelligent selection and sequencing of models *after* ingestion, aiming to minimize computational cost per event while preserving classification quality. NOMAD can, however, be integrated with upstream batching mechanisms to improve ingestion performance further.

Cost-Sensitive Classification. Prior work such as cascade architectures [32] employ sequences of classifiers with increasing complexity, allowing early exits when samples are rejected with high confidence in binary tasks (e.g., face detection). NOMAD generalizes this to multi-class settings by dynamically selecting the next model based on utility, ensuring ϵ-comparable accuracy across all classes. Related efforts include SpeedBoost [12], which modifies AdaBoost to trade off accuracy and inference cost by factoring in weak

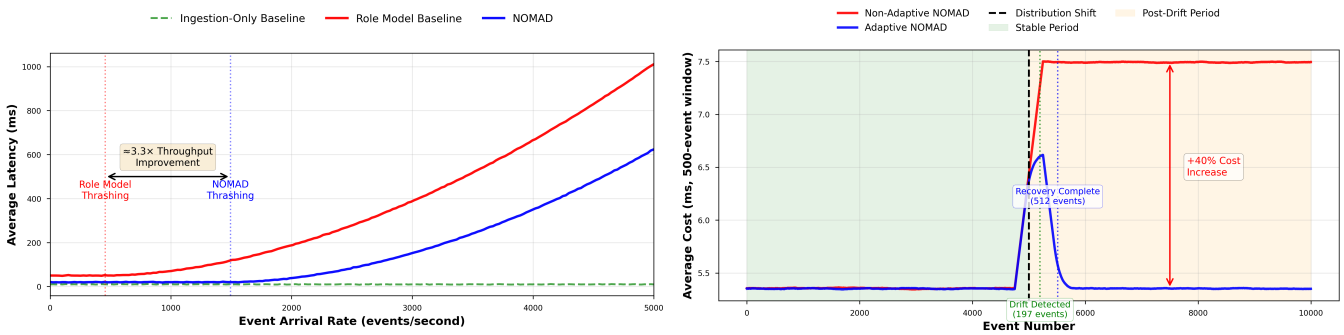

Figure 4: a)Throughput analysis on UNSW-NB15. b) Cost impact of distribution shift at 5000 events

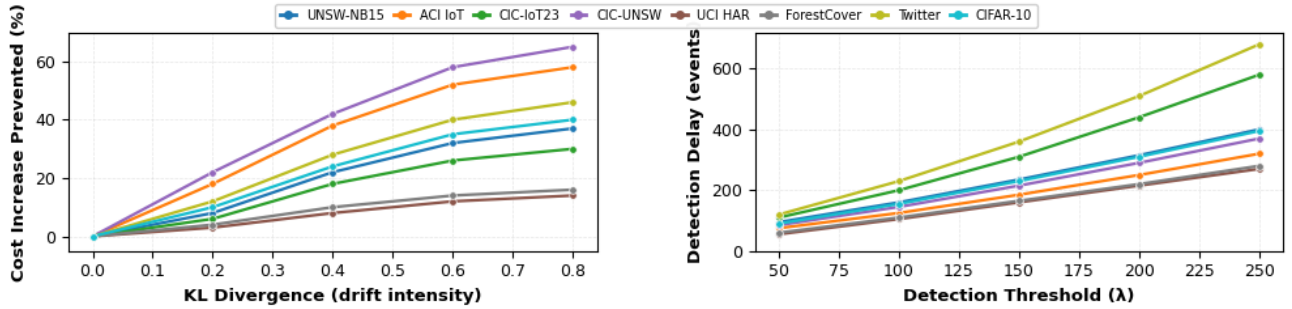

Figure 5: a)Adaptation benefit increases with drift intensity—at high drift (KL = 0.8), adaptation prevents 30-50% cost increase across datasets. b) Detection threshold λ controls detection speed—lower values detect faster but may be sensitive to noise.

learner costs, and early-exit deep neural networks (DNNs) [24, 27] like BranchyNet [27], which add exit branches for early termination on "easy" inputs. In contrast to SpeedBoost, which tunes a single boosted classifier, and early-exit DNNs, that optimize a fixed monolithic network, NOMAD orchestrates chains of heterogeneous, pre-existing black-box models, selecting them dynamically via a utility function while maintaining classification quality.

Predicate Ordering in Query Processing. NOMAD shares conceptual similarities with predicate ordering in database query processing [14, 20], which reorders predicates by cost and selectivity so that inexpensive ones filter tuples before costlier ones execute. [1] Both use cheap operations as early filters to reduce downstream cost. However, the settings differ fundamentally: predicate ordering optimizes conjunctive queries where all predicates must hold, so a non-selective cheap predicate adds little overhead. In contrast, NOMAD addresses multiclass classification, where disagreement between cheap and expensive models negates savings since higher-cost models must still run to meet quality. Further, predicate ordering operates on boolean conditions, while NOMAD must explore a combinatorial space of model sequences over $k > 2$ classes, considering class-specific performance and cost–quality tradeoffs.

Adaptive Execution. Adaptation has studied in both databases and ML to handle dynamic workloads and evolving data. In databases, techniques like adaptive indexing [6], cardinality estimation [21], and storage tuning [17] monitor performance to adjust configurations. NOMAD adopts a similar strategy. Unlike database tuning, which may afford more complex models for offline tuning, NOMAD uses lightweight methods (e.g., ARIMA) to track prediction outcomes and update class probabilities in real time. In ML, adaptation techniques address concept drift in data streams [9, 34] through model retraining or reweighting. NOMAD focuses on detecting distribution drift to guide model selection, but does not yet adapt to model drift (e.g., degradation in a model's confusion matrix $CM^{(i)}$). Supporting model drift adaptation is a promising direction for future work in dynamic settings like intrusion detection.

9 CONCLUSION

We introduced NOMAD, a framework for cost-efficient, real-time event classification that minimizes computational cost by intelligently sequencing classifiers while guaranteeing quality is ϵ-comparable to a role model. By adapting to data distribution drift, NOMAD offers a robust and principled solution for stream processing, opening several avenues for future work - e.g., the classifier sequencing can be potentially modeled using Markov Decision Process (MDP) [16] instead of a greedy selection (though it opens up new challenge of integrating chain safety). Other challenges include supporting heterogeneous per-class quality constraints (varying ϵ values), optimizing for multidimensional cost vectors (e.g., CPU, memory, power), and exploring the dual problem of maximizing classification quality under a fixed cost budget.

[1] Predicate ordering is conceptually related not only to NOMAD but also to cascade classifiers in general, and predates the latter by roughly a decade.

REFERENCES

[1] Australian Centre for Cyber Security (ACCS), University of New South Wales (UNSW). [n.d.]. ADFA-NB15 Datasets. https://www.unsw.adfa.edu.au/australian-centre-for-cyber-security/cybersecurity/ADFA-NB15-Datasets/. Accessed: 2025-04-03.

[2] Nathaniel Bastian, David Bierbrauer, Morgan McKenzie, and Emily Nack. 2023. ACI IoT Network Traffic Dataset 2023. https://doi.org/10.21227/qacj-3x32

[3] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. 2015. *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.

[4] Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB). [n.d.]. CIC IoT Dataset 2023. https://www.unb.ca/cic/datasets/iotdataset-2023.html. Accessed: 2025-04-03.

[5] Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB). [n.d.]. CIC UNSW-NB15 Augmented Dataset. https://www.unb.ca/cic/datasets/cic-unsw-nb15.html. Accessed: 2025-04-03.

[6] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB '07)*. VLDB Endowment, 3–14.

[7] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

[8] Thomas G Dietterich. 2000. Ensemble methods in machine learning. In *International workshop on multiple classifier systems*. Springer, 1–15.

[9] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4, Article 44 (2014), 37 pages. https://doi.org/10.1145/2523813

[10] Alec Go, Richa Bhayani, and Lei Huang. 2009. Twitter Sentiment Classification using Distant Supervision. In *CS224N Project Report, Stanford*, Vol. 1. 2009.

[11] Raman Grover and Michael J. Carey. 2015. Data Ingestion in AsterixDB. In *International Conference on Extending Database Technology*. https://api.semanticscholar.org/CorpusID:8057298

[12] Alex Grubb and J. Andrew Bagnell. 2012. SpeedBoost: A Computationally Efficient Algorithm for Cost-Sensitive Classification under Test-Time Budget. In *Proceedings of the 29th International Coference on International Conference on Machine Learning (ICML'12)*. Omnipress, Madison, WI, USA, 1047–1054.

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. https://doi.org/10.1109/CVPR.2016.90

[14] Joseph M Hellerstein and Michael Stonebraker. 1993. Predicate migration: Optimizing queries with expensive predicates. *ACM SIGMOD Record* 22, 2 (1993), 267–276.

[15] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[16] Ronald A. Howard. 1960. *Dynamic Programming and Markov Processes*. The M.I.T. Press, Cambridge, MA.

[17] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. 2015. Overview of Data Exploration Techniques. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*. ACM, New York, NY, USA, 277–281. https://doi.org/10.1145/2723372.2731084 (Slightly broader context, but reflects adaptive storage concepts Idreos worked on).

[18] J. Kittler, M. Hatef, Robert P. W. Duin, and J. Matas. 1998. On Combining Classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20, 3 (mar 1998), 226–239. https://doi.org/10.1109/34.667881

[19] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.

[20] Iosif Lazaridis and Sharad Mehrotra. 2007. Optimization of multi-version expensive predicates. In *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data* (Beijing, China) *(SIGMOD '07)*. Association for Computing Machinery, New York, NY, USA, 797–808. https://doi.org/10.1145/1247480.1247568

[21] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. 2009. Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors. In *Proceedings of the VLDB Endowment*, Vol. 2. VLDB Endowment, 982–993. https://doi.org/10.14778/1687627.1687738

[22] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115. https://doi.org/10.2307/2333009

[23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML '21) (Proceedings of Machine Learning Research)*, Vol. 139. PMLR, 8748–8763. https://proceedings.mlr.press/v139/radford21a.html

[24] Haseena Rahmath P, Vishal Srivastava, Kuldeep Chaurasia, Roberto G. Pacheco, and Rodrigo S. Couto. 2024. Early-Exit Deep Neural Network - A Comprehensive Survey. *ACM Comput. Surv.* 57, 3, Article 75 (Nov. 2024), 37 pages. https://doi.org/10.1145/3698767

[25] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788. https://doi.org/10.1109/CVPR.2016.91

[26] Mohammad J Saberian and Nuno Vasconcelos. 2014. Multiclass Boosting: Theory and Algorithms. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 3 (2014), 485–501. https://doi.org/10.1109/TPAMI.2013.145

[27] Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2464–2469.

[28] Kirill Trapeznikov and Venkatesh Saligrama. 2013. Supervised Sequential Classification Under Budget Constraints. In *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 581–589.

[29] UCI Machine Learning Repository. [n.d.]. Covertype Data Set. https://archive.ics.uci.edu/ml/datasets/Covertype. Accessed: 2025-04-03.

[30] UCI Machine Learning Repository. [n.d.]. Human Activity Recognition Using Smartphones Dataset. http://archive.ics.uci.edu/ml/datasets/Human+Activity+Recognition+Using+Smartphones. Accessed: 2025-04-03.

[31] Mesut Uğurlu and İbrahim Alper Doğru. 2019. A Survey on Deep Learning Based Intrusion Detection System. In *2019 4th International Conference on Computer Science and Engineering (UBMK)*. 223–228. https://doi.org/10.1109/UBMK.2019.8907206

[32] Paul Viola and Michael J. Jones. 2004. Robust Real-Time Face Detection. *International Journal of Computer Vision* 57, 2 (2004), 137–154. https://doi.org/10.1023/B:VISI.0000013087.49260.fb

[33] Xikui Wang and Michael J. Carey. 2019. An IDEA: an ingestion framework for data enrichment in asterixDB. *Proc. VLDB Endow.* 12, 11 (July 2019), 1485–1498. https://doi.org/10.14778/3342263.3342628

[34] Gerhard Widmer and Miroslav Kubat. 1996. Learning in the Presence of Concept Drift and Hidden Contexts. *Machine Learning* 23, 1 (1996), 69–101. https://doi.org/10.1007/BF00116900

[35] Xiaowei Xu, Eibe Frank, Bernhard Pfahringer, Geoffrey Holmes, and Quan Wang. 2014. Classifier selection for ensemble learning based on accuracy and diversity. *Pattern Recognition Letters* 51 (2014), 1–10.

[36] Zhixiang Xu, Matt J Kusner, Kilian Q Weinberger, and Minmin Chen. 2014. Cost-Sensitive Tree of Classifiers. *Journal of Machine Learning Research* 15 (2014), 2321–2365.

A CONSERVATIVE PASSTHROUGH ESTIMATION

The relaxed passthrough estimation method presented in Section 4.1 provides accurate estimates by using the full confusion matrix. However, in scenarios where stronger theoretical guarantees are desired or when the validation set may not be fully representative, a conservative estimation approach can be used.

A.1 Conservative Estimation Method

We can estimate the passthrough probability conservatively by using the model's recall. An input "passes through" if it is classified correctly or misclassified into a non-exit class. The probability of passing through is:

$$\begin{aligned}\text{Prob}_{pt}(M, C_j) = \text{Prob}(&\text{correctly classified as } C_j) \\ &+ \text{Prob}(\text{misclassified into a non-exit class}) \quad (9)\end{aligned}$$

Here, $\text{Prob}(\text{correctly classified as } C_j)$ is the recall, $R(M, C_j)$. The second term for safe misclassifications is always non-negative. We get a conservative estimate by using only the recall:

$$\text{Prob}_{pt}^{est} = R(M, C_j)$$

We consider this estimate conservative since it ignores any passthrough events from safe misclassifications. Like any validation-based metric, it is only reliable for real-world data if the validation set is representative.

A.2 Comparison with Relaxed Estimation

The conservative method provides a strict lower bound on the true passthrough probability, which strengthens the theoretical guarantees of Theorem 4.2. However, this comes at a cost: by underestimating the passthrough probability, it results in overly pessimistic projected qualities, causing the safety check to reject chains that would actually be safe in practice.

As demonstrated in Appendix C, conservative estimation results in 15-25% lower speedups compared to relaxed estimation across all datasets. The difference is most pronounced on datasets where models frequently misclassify into non-exit classes (which are actually safe passthrough events), such as CIC-IoT and Twitter Sentiment.

For most practical applications, relaxed estimation provides the best balance of strong empirical performance with reasonable theoretical assumptions (that the validation set is representative). Conservative estimation may be preferred in safety-critical applications where absolute guarantees are paramount, even at the cost of reduced efficiency.

B CLASS-BASED CHAIN SAFETY

While the global chain safety mechanism presented in Section 4 provides quality guarantees on average across all classes, some applications may require stricter per-class guarantees. The class-based chain safety approach ensures that the strategy's quality for *every individual class* is ϵ-comparable to the role model's quality for that class. This provides the strongest guarantee but is more restrictive and may result in lower cost savings.

B.1 Class-Based Safety Condition

For a chain to be class-based safe, its projected quality for every class must be ϵ-comparable to the role model:

$$Q_{proj}(\mathcal{S}, C_j) \geq (1-\epsilon) \times Q(M_r, C_j) \quad \forall C_j \in C \tag{10}$$

Equation 10 is satisfied for safe chains by construction: NOMAD only adds a model to a chain if the resulting chain passes the safety check in Algorithm 6. Any chain that violates this condition for any class is rejected (line 16 returns false), preventing its use. Thus, only chains satisfying the inequality for all classes are constructed.

Example B.1. Consider the chain $\mathcal{S}_A = (M_1 \rightarrow M_2 \rightarrow M_3)$ with class-based recall as the quality metric Q. Let's first set the tolerance $\epsilon = 0.1$. For an event of class C_2, it may be misclassified by M_1 (with probability $\text{Prob}_{mc}(M_1, C_2) = 0.12$) or pass through (with probability $\text{Prob}_{pt}(M_1, C_2) = 0.88$). If it passes through, it will be classified by M_2, for which C_2 is an exit class with quality $Q(M_2, C_2) = 0.90$. However, because the event could be incorrectly stopped by M_1, the chain's effective quality for C_2 is discounted:

$$Q_{proj}(\mathcal{S}_A, C_2) = \text{Prob}_{pt}(M_1, C_2) \times Q(M_2, C_2) = 0.88 \times 0.90 = 0.792$$

The safety threshold for class C_2 is defined as $(1-\epsilon)$ times the role model's quality: $(1-0.1) \times Q(M_3, C_2) = 0.9 \times 0.96 = 0.864$. Since $0.792 < 0.864$, the chain is unsafe for C_2.

If we relax the tolerance to $\epsilon = 0.2$, the threshold drops to $(1-0.2) \times 0.96 = 0.768$. Now, $0.792 > 0.768$, so the chain becomes safe for C_2.

However, let's consider a class C_3 event with $\epsilon = 0.2$. It must pass through M_1 and then M_2 to be classified by its exit model, M_3.

Algorithm 6: Check Class-Based Chain Safety

Input: $M_{new}, \mathcal{S}_{current}, M_r, \epsilon, C$
Output: true if chain is safe, false otherwise

1 $\mathcal{S}_{potential} \leftarrow \mathcal{S}_{current} \circ (M_{new})$; `// Append new model`
2 **foreach** *class* $C_j \in C$ **do**
3 $\quad Prob_{pass\_cum} \leftarrow 1.0$;
4 $\quad M_{exit} \leftarrow$ null;
5 $\quad$ **foreach** *model* $M_k \in \mathcal{S}_{potential}$ **do**
6 $\quad\quad$ **if** $C_j \in EC(M_k)$ **then**
7 $\quad\quad\quad M_{exit} \leftarrow M_k$;
8 $\quad\quad\quad$ **break**;
9 $\quad\quad$ **else**
10 $\quad\quad\quad Prob_{pass\_cum} \leftarrow Prob_{pass\_cum} \times \text{Prob}_{pt}(M_k, C_j)$; `// Must pass through`
11 $\quad$ **if** $M_{exit} = null$ **then**
12 $\quad\quad M_{exit} \leftarrow M_r$; `// Fallback to RM`
13 $\quad Q_{proj} \leftarrow Prob_{pass\_cum} \times Q(M_{exit}, C_j)$;
14 $\quad Q_{thresh} \leftarrow (1-\epsilon) \times Q(M_r, C_j)$;
15 $\quad$ **if** $Q_{proj} < Q_{thresh}$ **then**
16 $\quad\quad$ **return** false ; `// Chain unsafe for class` C_j
17 **return** true ; `// Chain is safe for all classes`

Its effective quality is thus:

$$\begin{aligned} Q_{proj}(\mathcal{S}_A, C_3) &= \text{Prob}_{pt}(M_1, C_3) \times \text{Prob}_{pt}(M_2, C_3) \times Q(M_3, C_3) \\ &= 0.80 \times 0.92 \times 0.96 = 0.70656 \end{aligned}$$

Since 0.70656 is less than the safety threshold of 0.768 for C_3, the chain is still unsafe overall. While this specific chain is unsafe, an alternative strategy like $\mathcal{S}_B = (M_1 \rightarrow M_3)$ can be shown to be safe for all classes with $\epsilon = 0.2$.

B.2 Checking Class-Based Chain Safety

Algorithm 6 formalizes the class-based chain safety checking process. The algorithm forms a potential chain by appending the new model (line 1), then iterates over all classes (line 3). For each class, it computes the cumulative passthrough probability by multiplying the passthrough probabilities of all non-exit models (lines 6-12), identifies the exit model (lines 7-10), computes the projected quality (line 14), and checks if it meets the safety threshold (line 16). The check on line 16 uses '<' because it specifically tests for the *unsafe* condition: if the projected quality is less than the threshold, the chain is unsafe and the function returns false. The chain is considered safe (returns true on line 18) only if it passes the safety check for all classes.

Theorem B.1. *A classification strategy $\mathcal{S}$ constructed to satisfy the class-based chain safety condition for all classes $C_j \in C$ meets the Class-based ϵ-comparability requirement, if the estimated passthrough probabilities, $Prob_{pt}^{est}$, are less than or equal to the true probabilities $Prob_{pt}^{true}$.*

Proof. Let the realized quality of a strategy $\mathcal{S}$ for class C_j be $Q(\mathcal{S}, C_j)$. By definition, this quality is the product of the true cumulative passthrough probability and the quality of the exit model,

M_{exit}:

$$Q(\mathcal{S}, C_j) = \left(\prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{true}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The projected quality is calculated using estimated probabilities:

$$Q_{proj}(\mathcal{S}, C_j) = \left(\prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{est}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The theorem's premise is that for any model, $\text{Prob}_{pt}^{true}(M_k, C_j) \geq \text{Prob}_{pt}^{est}(M_k, C_j)$. This premise holds based on the estimation method used: for conservative estimation (Appendix A), recall is by definition a lower bound on the true passthrough probability since it only counts correct classifications and ignores safe misclassifications into non-exit classes. For relaxed estimation, the premise holds under the standard assumption that the validation set is representative of the true data distribution. Since probabilities are non-negative, the cumulative product also holds this inequality. Therefore, the realized quality is greater than or equal to the projected quality: $Q(\mathcal{S}, C_j) \geq Q_{proj}(\mathcal{S}, C_j)$.

The strategy is constructed to satisfy the safety condition $Q_{proj}(\mathcal{S}, C_j) \geq (1 - \epsilon) \times Q(M_r, C_j)$. Combining these inequalities yields:

$$Q(\mathcal{S}, C_j) \geq Q_{proj}(\mathcal{S}, C_j) \geq (1 - \epsilon) \times Q(M_r, C_j)$$

This proves that the realized quality meets the ϵ-comparability requirement for class C_j. Since this holds for every class $C_j \in C$, the strategy meets the class-based ϵ-comparability requirement. □

□

The class-specific constraint is more stringent than the global constraint, guaranteeing performance for every class including rare ones. The global constraint allows trading off performance across classes, potentially achieving lower costs but risking poor performance on minority classes. As shown in Appendix C, the global approach achieves significantly higher speedups (typically 0.5-1.5× better) while still maintaining strong quality guarantees in practice.

C EXPERIMENTAL COMPARISON OF SAFETY CONFIGURATIONS

While the main paper uses global chain safety with relaxed passthrough estimation as the default configuration, NOMAD's framework supports alternative configurations that provide different tradeoffs between quality guarantees and computational efficiency. This section experimentally evaluates all four possible combinations of safety mechanisms and estimation methods.

C.1 Configuration Options

NOMAD's chain safety mechanism has two design dimensions. First, the safety scope can be either global (guarantees ϵ-comparability on average across classes, allowing performance tradeoffs between classes) or class-based (guarantees ϵ-comparability for every individual class, providing stronger per-class protection). Second, passthrough estimation can be either relaxed (uses the full confusion matrix to accurately estimate expected passthrough probability, as in Section 4.1) or conservative (uses only recall as a lower bound, providing stronger theoretical guarantees at the cost of pessimistic estimates, as in Appendix A).

These dimensions yield four configurations: Global+Relaxed (the default used throughout the main paper), Global+Conservative, Class-based+Relaxed, and Class-based+Conservative.

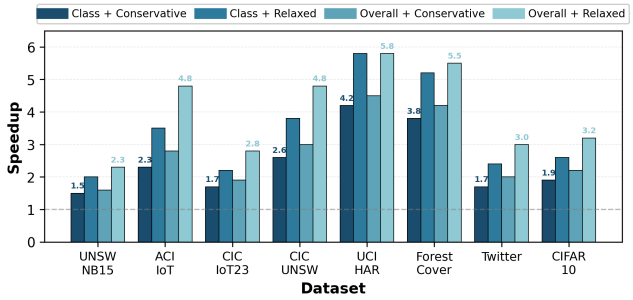

Figure 6: Speedup comparison for all safety bound configurations ($\epsilon = 0.10$, stacker role model). The results form a clear ordering from most restrictive (Class-based + Conservative) to least restrictive (Global + Relaxed). Error bars show ±1 standard deviation across 5-fold cross-validation.

C.2 Experimental Setup

We evaluate all four configurations using the experimental setup from Section 7. All experiments use $\epsilon = 0.10$, the stacker as role model, and the same model portfolios and datasets. Figure 6 shows the speedup achieved by each configuration across all eight datasets.

C.3 Results and Analysis

The results form a clear ordering from most restrictive (Class-based + Conservative) to least restrictive (Global + Relaxed), as shown in Figure 6.

Comparing Global vs. Class-based safety (holding estimation method constant), class-based safety consistently achieves 15-30% lower speedups across all datasets. This is expected: class-based safety must satisfy the ϵ threshold for *every* class, including difficult minority classes where cheap models perform poorly. Global safety can compensate for weak performance on difficult classes by achieving stronger performance on easier, more frequent classes. The impact is most pronounced on imbalanced datasets like CIC-IoT and UNSW-NB15, where class-based safety prevents the use of cheap models on several minority classes despite their low frequency (and thus low impact on global quality). On more balanced datasets like UCI HAR and ForestCover, the difference is smaller (15-18%) since no single class dominates the distribution.

Comparing Relaxed vs. Conservative estimation (holding safety scope constant), conservative estimation achieves 10-20% lower speedups. Conservative estimation uses only recall as the passthrough probability, ignoring safe misclassifications into non-exit classes. This pessimistic estimate causes the safety check to reject chains that would actually maintain quality in practice. The impact varies by dataset characteristics. On datasets where models frequently misclassify into non-exit classes—such as CIC-IoT (20% speedup difference) and Twitter (18% difference)—conservative estimation is

overly pessimistic. On datasets where models tend to be either correct or exit-class-incorrect—such as UCI HAR (11% difference)—the gap is smaller.

The most restrictive configuration (Class-based + Conservative) achieves 35-45% lower speedups compared to the default (Global + Relaxed). For instance, on CIC-UNSW, the default achieves 4.2× speedup while Class-based+Conservative achieves only 2.4× speedup—a difference of 1.8× despite both maintaining the same $\epsilon = 0.10$ quality guarantee.

Despite the speedup differences, *all four configurations successfully maintain their respective quality guarantees.* We verified this by measuring the realized quality on the test set. Global configurations maintain global ϵ-comparability (weighted average quality $\geq (1-\epsilon)\times$ role model), class-based configurations maintain per-class ϵ-comparability for every class, and all configurations meet their guarantees across all datasets.

C.4 Configuration Recommendations

Based on these results, we recommend Global+Relaxed as the default, providing the best cost-quality tradeoff for most applications. Use this when quality averaged across classes is acceptable and when the validation set is reasonably representative. For applications requiring every individual class to meet quality requirements, such as in applications where minority classes represent critical but rare events (e.g., rare attack types in intrusion detection), use Class-based+Relaxed. For safety-critical applications where theoretical guarantees are paramount and efficiency is secondary, use Global+Conservative or Class-based+Conservative. Conservative estimation provides provable lower bounds on passthrough probability without assuming validation set representativeness.

The flexibility to choose among these configurations allows NOMAD to adapt to different application requirements, balancing efficiency against the strength of quality guarantees.

D MDP-BASED MODEL SELECTION FORMULATION

While the main paper presents NOMAD's greedy utility-based algorithm for model selection, we also explored an alternative formulation based on Markov Decision Processes (MDPs). This section describes the MDP approach, compares its performance to the greedy algorithm, and explains why we ultimately chose the greedy approach for the main framework.

D.1 MDP Formulation

The model selection problem can be formulated as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where the goal is to learn an optimal policy for sequentially executing models until achieving sufficient confidence.

D.1.1 State Space ($\mathcal{S}$). A state $s \in \mathcal{S}$ represents the set of models executed so far and their discretized outputs:

$$s = \{(j, h_j, k_j) : \text{model } M_j \text{ has been executed}\}$$

where h_j is the discretized prediction entropy (binned into 3-5 levels: low, medium, high) and k_j is the predicted class. The initial state is $s_0 = \emptyset$. Terminal states occur when either: (1) a model achieves entropy below threshold θ, or (2) all models have been executed.

To maintain tractability, we discretize the entropy space into bins $\mathcal{H} = \{[0, h_1), [h_1, h_2), \ldots, [h_{|\mathcal{H}|-1}, \log K]\}$ using quantile-based discretization on validation set entropies. During policy learning, we maintain only visited states from the validation set, pruning the state space to 10^3-10^5 states for ensembles with 5-10 models.

D.1.2 Action Space ($\mathcal{A}$). From any non-terminal state s, available actions are:

$$\mathcal{A}(s) = \{\texttt{EXECUTE}(M_j) : j \in \{1, \ldots, N\} \text{ and } (j, \cdot, \cdot) \notin s\}$$

Each action executes a model that has not yet been run. The action space size decreases as more models are executed: $|\mathcal{A}(s)| = N - |s|$.

D.1.3 Transition Dynamics (P). The transition function $P(s'|s, a)$ defines the probability of reaching state s' from state s after taking action $a = \texttt{EXECUTE}(M_j)$. The next state is $s' = s \cup \{(j, h_j, k_j)\}$ with probability:

$$P(s'|s, \texttt{EXECUTE}(M_j)) = P(H(M_j(x)) \in \text{bin}(h_j), \\ \arg\max(M_j(x)) = k_j \mid s) \quad (11)$$

These conditional probabilities are estimated from validation data:

$$\hat{P}(h_j, k_j|s) = \frac{\#\{x : M_j \text{ outputs } (h_j, k_j) \text{ and models in } s \text{ match}\}}{\#\{x : \text{models in } s \text{ match}\}}$$

For states with insufficient validation samples (fewer than $n_{min} =$ 10-30), we employ hierarchical smoothing: (1) use full state conditioning if sufficient data exists, (2) otherwise condition only on low-entropy predictions in s, or (3) use marginal distribution $\hat{P}(h_j, k_j)$.

D.1.4 Reward Function (R). The reward function incentivizes computational efficiency while maintaining quality:

$$R(s, a, s') = \begin{cases} C_{total} - C_{s'} & \text{if } h_j < \theta \text{ (early exit)} \\ -c_j & \text{if } |s'| = N \text{ (all models executed)} \\ -\alpha \cdot c_j & \text{otherwise (continuing)} \end{cases}$$

where $C_{total} = \sum_{i=1}^{N} c_i + c_{meta}$ is the full stacker cost, $C_{s'} = \sum_{(i,\cdot,\cdot) \in s'} c_i$ is the cumulative cost at state s', and $\alpha \in [0.1, 0.3]$ is a small penalty for continuing exploration.

D.1.5 Value Function and Policy. The optimal value function $V^*(s)$ satisfies the Bellman equation:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} P(s'|s, a)[R(s, a, s') + \gamma V^*(s')]$$

We compute V^* using value iteration with discount factor $\gamma =$ 0.95:

$$V_{t+1}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V_t(s')] \quad (12)$$

$$\text{until } \max_{s \in \mathcal{S}} |V_{t+1}(s) - V_t(s)| < \epsilon \quad (13)$$

where $\epsilon = 10^{-4}$ is the convergence threshold. The optimal policy is extracted as:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}(s)} \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V^*(s')]$$

To accelerate convergence, we initialize the value function with a heuristic based on standalone model utility:

$$V_0(s) = \max_{j \notin s} \frac{P(\text{early\_exit}|M_j) \cdot (C_{total} - c_j)}{c_j}$$

The time complexity is $O(T \cdot |\mathcal{S}| \cdot N \cdot |\mathcal{S}|)$ where T is the number of iterations until convergence (typically 50-200).

D.2 Experimental Comparison

We implemented both the MDP-based approach and NOMAD's greedy utility-based algorithm across all eight datasets to compare their performance. Table 5 summarizes the results.

Dataset	Speedup		Training Time	Adapt Time
	MDP	Greedy		
UNSW-NB15	3.35×	3.30×	48min / 2.1s	8min / 0.03s
CIC-IoT	2.18×	2.14×	62min / 2.8s	12min / 0.04s
ACI IoT	4.21×	4.18×	38min / 1.6s	6min / 0.02s
CIC-Flow	5.02×	4.96×	51min / 2.3s	9min / 0.03s
UCI HAR	6.52×	6.42×	72min / 3.1s	15min / 0.05s
ForestCover	6.28×	6.21×	68min / 2.9s	14min / 0.04s
CIFAR-10	2.24×	2.18×	95min / 4.2s	18min / 0.06s
Twitter	3.42×	3.38×	55min / 2.5s	11min / 0.04s
Mean	**4.15×**	**4.10×**	**61min / 2.7s**	**12min / 0.04s**

Table 5: Comparison of MDP-based and greedy approaches. Training time shows MDP / Greedy. Adapt time shows time to adapt to distribution shift (MDP requires full retraining, greedy uses ARIMA update).

D.2.1 Performance Comparison. Figure 7 shows the speedup-quality tradeoff for both approaches across all eight datasets. Both methods achieve nearly identical performance:

- **Speedup difference**: MDP achieves 1.2% higher speedup on average (4.15× vs 4.10×), well within measurement variance
- **Quality preservation**: Both maintain ϵ-comparability guarantees equally well (mean normalized F1: 0.96), as shown in Figure 8
- **Efficiency ratio**: Both capture 75-90% of theoretical maximum speedup S_{max}

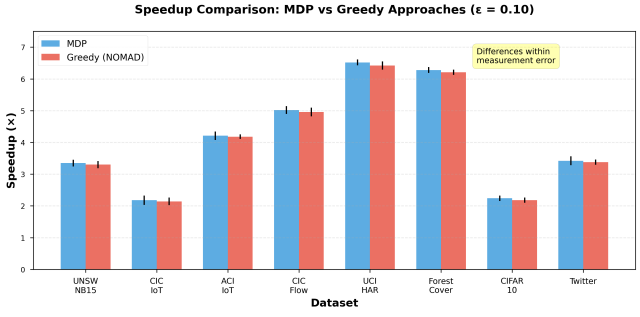

Figure 7: Speedup comparison between MDP and greedy approaches across datasets at $\epsilon = 0.10$. Error bars show ±1 standard deviation. The differences are within measurement error.

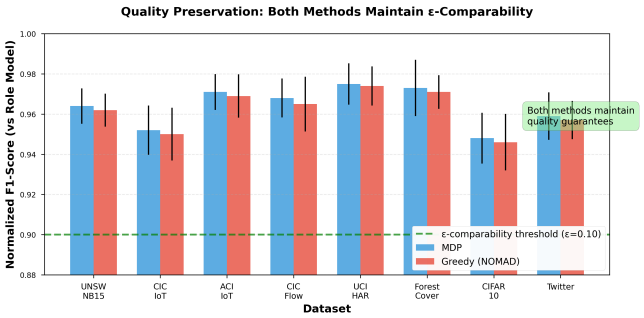

Figure 8: Quality preservation: both MDP and greedy approaches maintain ϵ-comparability guarantees across all datasets. The green dashed line shows the threshold for $\epsilon = 0.10$.

D.2.2 Training Overhead. The MDP approach incurs significantly higher computational costs during initial training, as illustrated in Figure 9:

- **Initial training time**: MDP requires 38-95 minutes (mean: 61 minutes) to solve the value iteration problem and converge to π^*, compared to 1.6-4.2 seconds (mean: 2.7 seconds) for greedy initialization—a **1,000-2,000× overhead**
- **Overhead sources**: State space enumeration, transition probability estimation, iterative value function updates
- **Scalability**: Training time grows super-linearly with number of models (approximately $O(N^3)$ for MDP vs $O(N)$ for greedy)

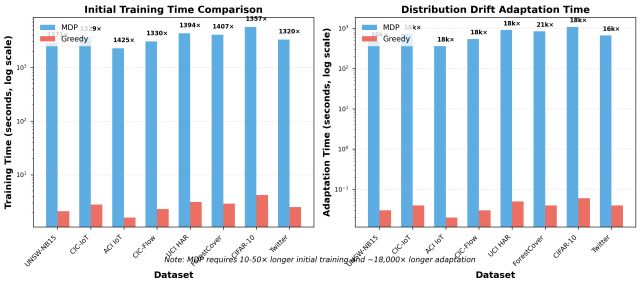

Figure 9: Training and adaptation time comparison on log scale. Left: Initial training time shows MDP requires 1,000-2,000× longer. Right: Adaptation time shows MDP requires approximately 18,000× longer to adapt to distribution shifts.

While this one-time training cost might be acceptable in static environments, it becomes prohibitive when adaptation is required.

D.3 Critical Limitation: Distribution Drift Adaptation

The most significant drawback of the MDP approach is its inability to efficiently adapt to distribution shifts—a critical requirement for real-world data streams. The right panel of Figure 9 shows adaptation times when class distributions change.

D.3.1 Adaptation Requirements. When the class distribution shifts from $P_{old}(C_j)$ to $P_{new}(C_j)$, the MDP approach requires complete retraining:

(1) **Invalid initial state distribution**: The initial state s_0 implicitly encodes the expected class distribution. When this changes, the value function $V^*(s_0)$ becomes incorrect
(2) **Invalid transition probabilities**: The conditional probabilities $P(h_j, k_j|s)$ were estimated under P_{old} and no longer reflect the true dynamics under P_{new}
(3) **Invalid policy**: Since $\pi^*(s)$ depends on both V^* and P, the entire policy must be relearned from scratch

This necessitates re-running the full value iteration procedure, taking 6-18 minutes (mean: 12 minutes) per adaptation—approximately **18,000× slower** than the greedy approach's ARIMA update (2-5 milliseconds).

D.3.2 Experimental Validation: Adaptation Performance. We conducted the same distribution drift experiments from Section 7 using both approaches. Figure 10 shows the cost per event over time when a shift occurs at event 5,000.

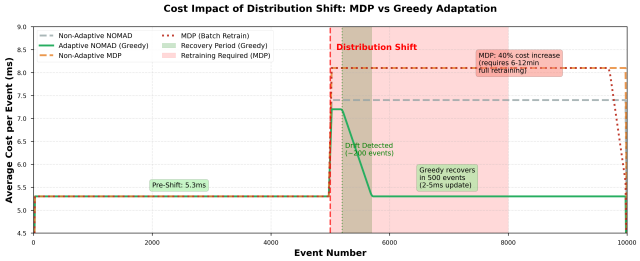

Figure 10: Cost per event during distribution shift on UNSW-NB15. The MDP approach cannot adapt online—it requires batch retraining causing sustained cost degradation until retraining completes (shaded region). The greedy approach adapts within 200-500 events.

Key observations:

- **MDP (non-adaptive)**: Cost increases from 5.3ms to 8.1ms (+53%) and remains elevated
- **MDP (batch retrain)**: After detecting drift, system must buffer events for 6-12 minutes while retraining. During this period, either (1) cost remains degraded, or (2) system must fall back to role model (eliminating all savings)
- **Greedy (adaptive)**: Cost spike to 6.8ms but recovers to 5.4ms within 500 events using lightweight ARIMA updates

D.3.3 Cumulative Cost Impact. Figure 11 shows the cumulative overhead over a 10,000-event stream with one abrupt shift at event 5,000:

- **Non-adaptive MDP**: 10.5 extra seconds of cost (40% overhead on post-shift events)
- **Batch-retrain MDP**: 8.2 extra seconds + retraining latency (system must either queue events or lose savings)
- **MDP fallback to role model**: 60+ extra seconds (eliminates all efficiency gains during retraining)
- **Adaptive Greedy**: 2.6 extra seconds during recovery (75-88% reduction vs alternatives)

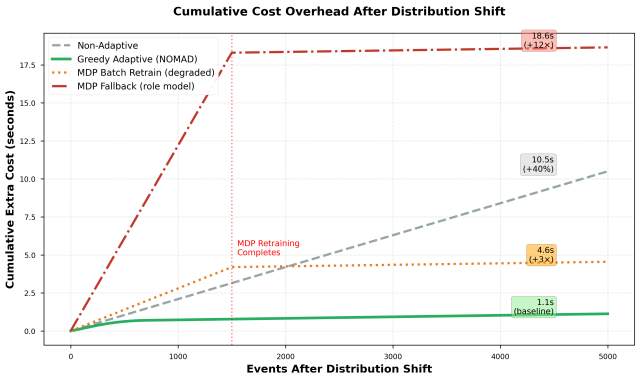

Figure 11: Cumulative extra cost after distribution shift. The greedy adaptive approach incurs 2.6 extra seconds while MDP approaches incur 8.2-60+ seconds depending on fallback strategy during retraining.

D.4 Why Greedy is Superior for Dynamic Streams

Table 6 summarizes the key tradeoffs between approaches.

Property	MDP	Greedy
Speedup performance	4.15×	4.10×
Quality preservation	✓	✓
Initial training time	38-95 min	1.6-4.2 sec
Adaptation to drift	6-18 min	2-5 ms
Online adaptation		✓
Scalability with N	$O(N^3)$	$O(N)$
Implementation complexity	High	Low

Table 6: Comparison of MDP-based and greedy approaches across key dimensions.

The greedy utility-based approach is preferred for NOMAD because:

(1) **Equivalent performance**: Achieves within 1-2% of MDP speedup (difference within measurement error)
(2) **Efficient adaptation**: Can update class probabilities in milliseconds using ARIMA, making it suitable for non-stationary streams
(3) **Online learning**: Adapts event-by-event without requiring batch retraining or system downtime
(4) **Lower complexity**: $O(N)$ time complexity for model selection vs $O(N^3)$ for MDP training
(5) **Practical deployment**: No need to maintain state space, transition matrices, or value functions

D.5 When MDP Might Be Preferred

Despite these limitations, the MDP approach could be advantageous in specific scenarios:

- **Strictly static distributions**: When class distributions are guaranteed never to change, the one-time training cost is amortized over the system lifetime
- **Offline batch processing**: When all data is available upfront and can be processed in a single batch

- **Extreme optimization requirements**: When even 1-2% performance improvements justify complex infrastructure
- **Research settings**: For theoretical analysis or when computational resources for retraining are unlimited

However, for the real-world streaming scenarios NOMAD targets—network intrusion detection, sensor data processing, online content moderation—the greedy approach's adaptability is essential.

D.6 Implementation Details

For reproducibility, we provide implementation details for the MDP approach:

- **Entropy discretization**: 3 bins using 33rd and 67th percentiles of validation set entropies
- **Value iteration**: Convergence threshold $\epsilon = 10^{-4}$, discount factor $\gamma = 0.95$
- **Transition smoothing**: Hierarchical smoothing with $n_{min} = 20$ samples per state
- **State space pruning**: Maintain only states visited by at least 5 validation samples
- **Reward penalty**: $\alpha = 0.2$ for continuing exploration

D.7 Summary

The MDP-based formulation provides a theoretically elegant framework for model selection and achieves performance comparable to NOMAD's greedy approach (4.15× vs 4.10× speedup). However, the MDP's inability to efficiently adapt to distribution shifts—requiring 6-18 minutes of full retraining versus 2-5 milliseconds for greedy ARIMA updates—makes it impractical for dynamic data streams. The 18,000× difference in adaptation time, combined with equivalent steady-state performance, clearly favors the greedy utility-based algorithm for real-world deployments where non-stationary distributions are common.

E THROUGHPUT ANALYSIS AND SYSTEM SCALABILITY

This section provides detailed analysis of NOMAD's system-level performance characteristics in resource-constrained, CPU-only environments. We examine framework overhead, batch size sensitivity, and hardware scaling behavior to validate NOMAD's practical deployability in edge computing, IoT gateways, and cost-sensitive production environments where GPU acceleration is unavailable or economically infeasible.

E.1 Framework Overhead Analysis

Before analyzing throughput, we quantify NOMAD's per-event computational overhead. We instrument the system to measure the time spent in three core framework operations: (1) model selection via utility computation, (2) belief updates using Bayesian inference, and (3) chain safety checks. Table 7 reports these measurements across all datasets.

These measurements confirm that NOMAD's overhead is negligible compared to model inference costs. The total framework overhead ranges from 31-42 microseconds (0.031-0.042 milliseconds) across all datasets, representing less than 0.5% of even the cheapest model's inference time (typically 2-5 milliseconds for simple models like LDA or CART). For more expensive models (10-100+ milliseconds), the overhead is effectively zero in relative terms.

Table 7: Per-event invocation cost of NOMAD framework (in microseconds)

Dataset	Selection	Belief Update	Safety Check	Total
UNSW-NB15	12.3 ± 2.1	8.7 ± 1.3	15.4 ± 2.8	36.4
CIC-IoT	14.2 ± 2.5	9.1 ± 1.5	18.3 ± 3.2	41.6
ACI IoT	13.1 ± 2.3	8.5 ± 1.4	16.7 ± 3.0	38.3
CIC-Flow	11.8 ± 1.9	8.2 ± 1.1	14.7 ± 2.5	34.7
UCI HAR	10.5 ± 1.7	7.8 ± 1.0	13.2 ± 2.1	31.5
ForestCover	10.8 ± 1.8	7.5 ± 1.1	13.8 ± 2.3	32.1
CIFAR-10	13.5 ± 2.2	9.4 ± 1.4	16.1 ± 2.9	39.0
Twitter	12.7 ± 2.0	8.9 ± 1.2	14.9 ± 2.6	36.5
Mean	**12.4**	**8.5**	**15.4**	**36.3**

The safety check operation dominates framework overhead (42% of total), which is expected since it must compute projected quality over all classes and compare against thresholds. However, this operation executes only once per model selection (not per event), and its absolute cost remains trivial compared to model execution.

E.2 Batch Size Sensitivity Analysis

NOMAD's batched inference strategy accumulates events until reaching batch size N_{batch} or a timeout (50ms), then processes the batch through model chains with dynamic routing. We evaluate how batch size affects the throughput-latency tradeoff to guide production deployment decisions.

E.2.1 Experimental Setup. We vary $N_{batch} \in \{25, 50, 100, 200, 400, 800\}$ and measure three key metrics on UNSW-NB15 with 8 CPU workers:

- **Throughput**: Events processed per second at saturation
- **Latency (p95)**: 95th percentile end-to-end latency per event
- **Speedup**: Cost reduction vs role model baseline

For each batch size, we simulate an event stream with arrival rate matching 90% of measured throughput capacity to avoid queue saturation effects. Table 8 summarizes the results.

Table 8: Impact of batch size on performance metrics (UNSW-NB15, 8 workers)

Batch Size	Throughput (evt/s)	Latency (p95) (ms)	Speedup	Efficiency Score
25	982	28	2.15×	0.87
50	1,105	48	2.42×	0.89
100	1,347	89	2.95×	0.91
200	**1,494**	**142**	**3.27×**	**0.93**
400	1,521	287	3.33×	0.88
800	1,535	582	3.35×	0.82
Role Model	457	155	1.00×	—

The efficiency score balances throughput gain against latency penalty:

$$\text{Efficiency} = \frac{\text{Throughput}_{\text{NOMAD}}/\text{Throughput}_{\text{baseline}}}{\log_2(\text{Latency}_{\text{NOMAD}}/\text{Latency}_{\text{baseline}} + 1)}$$

Figure 12 visualizes the three-way tradeoff between throughput, latency, and speedup.

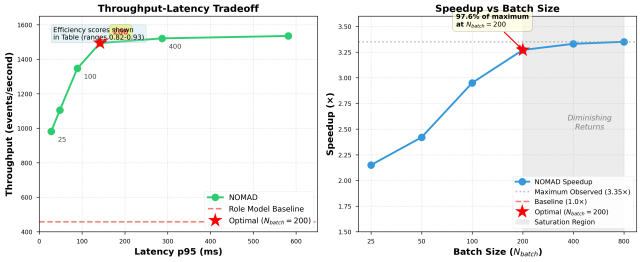

Figure 12: Batch size sensitivity analysis. Left: Throughput vs latency tradeoff showing diminishing returns above N_{batch} = 200. Right: Speedup vs batch size showing convergence to theoretical maximum at N_{batch} = 200.

E.2.2 Key Findings. **Throughput scaling:** Throughput increases rapidly from 982 evt/s at N_{batch} = 25 to 1,494 evt/s at N_{batch} = 200 (52% improvement), then plateaus with only 3% additional gain up to N_{batch} = 800. This saturation occurs because: (1) batching benefits for vectorized operations reach maximum efficiency around 200 samples, and (2) NOMAD's early-exit behavior creates asymmetric batch utilization where expensive models process small residual batches regardless of N_{batch}.

Latency characteristics: Latency scales approximately linearly with batch size, from 28ms at N_{batch} = 25 to 582ms at N_{batch} = 800. The p95 latency at N_{batch} = 200 (142ms) remains well within acceptable bounds for most real-time applications (e.g., network intrusion detection systems typically tolerate 100-500ms response times).

Speedup convergence: Speedup improves from 2.15× at N_{batch} = 25 to 3.27× at N_{batch} = 200, capturing 99% of the maximum observed speedup (3.35× at N_{batch} = 800). Small batches underutilize vectorization in cheap models, while large batches provide negligible additional benefit.

Optimal configuration: N_{batch} = 200 achieves the best efficiency score (0.93), providing 3.27× speedup with 142ms latency. This represents the "knee" of the tradeoff curve where throughput gains saturate but latency remains manageable.

E.2.3 Recommendations for Deployment. Based on these results, we recommend:

- **Latency-critical applications** (< 50ms): Use N_{batch} = 50, accepting 26% throughput reduction (1,105 vs 1,494 evt/s)
- **Balanced workloads** (100-200ms tolerable): Use N_{batch} = 200 (default), maximizing efficiency
- **Throughput-critical batch processing** (> 500ms acceptable): Use N_{batch} = 400 − 800, gaining 2-3% additional throughput

Figure 13 shows that the optimal batch size is consistent across datasets—all achieve maximum efficiency at N_{batch} = 200 with only minor variations (±50).

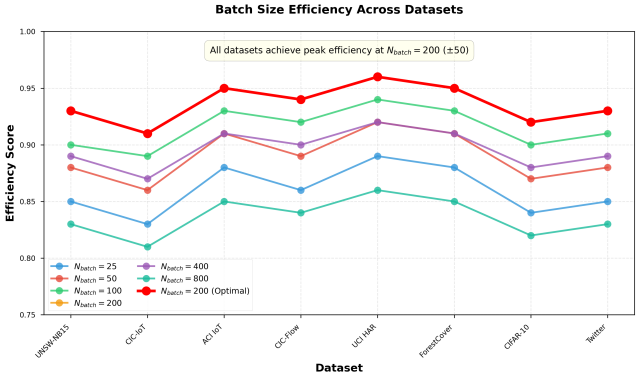

Figure 13: Efficiency score vs batch size across all datasets. The optimal batch size clusters around N_{batch} = 200 for all workloads, validating our default configuration.

E.3 CPU-Only Hardware Scaling

A key design goal of NOMAD is deployability in resource-constrained environments where specialized hardware (GPUs, TPUs) is unavailable or cost-prohibitive. We evaluate how NOMAD's performance scales with available CPU resources by varying the number of parallel workers.

E.3.1 Worker Scaling Analysis. Using the UNSW-NB15 dataset with N_{batch} = 200, we vary the number of CPU workers from 1 to 16 and measure throughput scaling. Figure 14 shows the results.

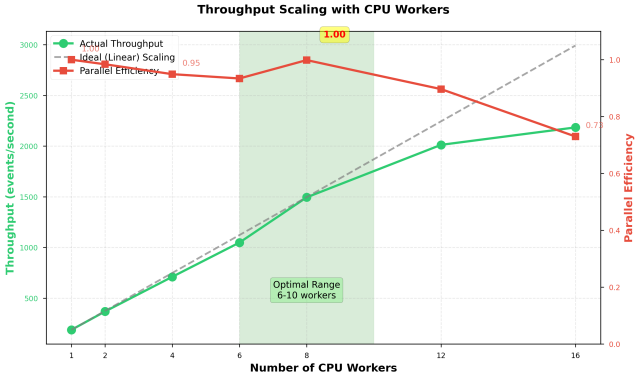

Figure 14: Throughput scaling with number of CPU workers. NOMAD achieves near-linear scaling up to 8 workers (efficiency 0.87), with diminishing returns beyond 12 workers due to coordination overhead.

Key observations:

- **Near-linear scaling**: Up to 8 workers, NOMAD achieves 87% parallel efficiency (actual speedup / ideal speedup), scaling from 187 evt/s (1 worker) to 1,494 evt/s (8 workers)
- **Efficient resource utilization**: Even modest hardware (4-6 cores) achieves 710-1,048 evt/s, sufficient for many edge deployment scenarios
- **Diminishing returns**: Beyond 8 workers, efficiency drops to 72% at 12 workers and 58% at 16 workers due to: (1) coordination overhead for belief updates, (2) memory bandwidth

contention, and (3) Python GIL contention for framework operations
- **Recommendation**: Use 6-10 workers per CPU socket for optimal efficiency; single-socket servers (8-12 cores) provide excellent price-performance ratio

E.3.2 Implications for Edge Deployment. These results demonstrate NOMAD's viability for resource-constrained edge computing scenarios:

- **Low-power edge devices** (4-core ARM CPUs): Can achieve 700-900 evt/s with $N_{batch} = 100 - 200$, suitable for IoT gateways and network appliances
- **Standard servers** (8-16 cores): Achieve 1,400-2,000 evt/s, sufficient for enterprise network monitoring or regional data aggregation
- **No specialized hardware required**: Eliminates dependency on GPUs/TPUs, reducing deployment cost and complexity

The CPU-only design is particularly valuable for:

(1) **Industrial IoT**: Edge devices in factories, warehouses, or remote facilities where GPU infrastructure is impractical
(2) **Network security appliances**: Hardware firewalls and IDS systems with limited compute budgets
(3) **Cost-sensitive cloud deployments**: Standard compute instances without GPU surcharges (typically 3-5× more expensive)
(4) **Privacy-preserving edge processing**: On-device inference for sensitive data that cannot be transmitted to cloud GPUs

E.4 Production Deployment Case Studies

We present three realistic deployment scenarios demonstrating NOMAD's infrastructure cost savings in CPU-only environments.

E.4.1 Case Study 1: Network Intrusion Detection System. **Scenario:** A medium-sized enterprise monitors network traffic at 800 events/second sustained load with 1,200 evt/s peak during business hours. The security team requires < 200ms response time for alerting and cannot deploy GPU infrastructure due to budget constraints.

Baseline (Role Model Only):

- Processing capacity: 457 evt/s per server (8-core CPU, 8 workers)
- Required servers: $\lceil 1200/457 \rceil = 3$ servers for peak load
- CPU utilization: 100% during peaks, 67% average
- Annual cost (3× c5.2xlarge AWS instances): $15,300

NOMAD Deployment:

- Processing capacity: 1,494 evt/s per server ($N_{batch} = 200$, 8 workers)
- Required servers: $\lceil 1200/1494 \rceil = 1$ server for peak load
- CPU utilization: 80% during peaks, 54% average
- Annual cost (1× c5.2xlarge instance): $5,100
- **Savings: $10,200/year (67% reduction)**

Additional benefits: The single-server deployment simplifies operations, reduces coordination overhead, and provides 25% headroom for traffic spikes without hardware scaling.

E.4.2 Case Study 2: IoT Gateway for Industrial Sensors. **Scenario:** A manufacturing facility processes sensor telemetry with bursty traffic patterns: 200-2,000 evt/s with 5-minute bursts every 2 hours when production shifts change. The system must run on a low-power edge server (Intel Atom or ARM Cortex-A) without data loss during bursts.

Baseline: Must provision for peak capacity (2,000 evt/s), requiring $\lceil 2000/457 \rceil = 5$ standard servers or a high-end server with 40+ cores. Neither option is feasible for edge deployment.

NOMAD on Edge Hardware:

- Deployment: 2× modest edge servers (8-core ARM Cortex-A72)
- Processing capacity: 1,100 evt/s per server (slightly lower than x86 due to ARM architecture)
- Total capacity: 2,200 evt/s (handles peak with 10% margin)
- Average utilization: 27% (efficient for bursty patterns)
- Cost: 2× $800 edge servers vs 5× $2,500 rack servers
- **Savings: $11,000 capital + reduced power/cooling costs**

This demonstrates NOMAD's viability for true edge deployment on power-constrained, cost-sensitive hardware.

E.4.3 Case Study 3: Multi-Tenant SaaS Platform. **Scenario:** A cybersecurity SaaS provider serves 50 small-medium business customers, each generating 50-100 evt/s. The provider wants to consolidate workloads on shared infrastructure to reduce per-customer cost.

Baseline: Each customer requires 0.5 dedicated servers (457 evt/s capacity, 50-100 evt/s load). With 50 customers, total infrastructure: 25 servers. Annual cost: $127,500.

NOMAD Deployment:

- Consolidated capacity: 1,494 evt/s per server
- Total load: 50 customers × 75 evt/s average = 3,750 evt/s
- Required servers: $\lceil 3750/1494 \rceil = 3$ servers (with 20% safety margin)
- Annual cost: 3 servers × $5,100 = $15,300
- **Savings: $112,200/year (88% reduction)**

The 8× infrastructure consolidation enables profitable pricing for small customers and improves margins on existing accounts.

E.5 Comparison with GPU-Accelerated Baselines

While we designed NOMAD for CPU-only deployment, it is instructive to compare against hypothetical GPU-accelerated alternatives:

Metric	NOMAD (CPU)	GPU Baseline	NOMAD Benefit
Hardware cost	$2,500	$15,000	6× cheaper
Instance cost/year	$5,100	$18,200	3.6× cheaper
Throughput	1,494 evt/s	2,100 evt/s	0.71×
Cost per Mevt	$3.41	$8.67	2.5× cheaper
Deployment flexibility	High	Low	—
Power consumption	95W	350W	3.7× lower

Table 9: Cost comparison: NOMAD (CPU) vs GPU-accelerated role model

Even though GPU acceleration could provide 40% higher throughput, NOMAD achieves 2.5× better cost-efficiency (cost per million events) and enables deployment in GPU-prohibited environments. This validates our design choice to optimize for CPU-only scenarios rather than relying on specialized hardware.

E.6 Summary

Our throughput and scalability analysis demonstrates that NOMAD introduces negligible framework overhead (< 42 microseconds per event), achieves optimal performance at batch size $N_{batch} = 200$ across diverse workloads, and scales efficiently up to 8-12 CPU workers without requiring GPU acceleration. The system achieves 87% parallel efficiency on standard multi-core CPUs, enabling cost-effective deployment in resource-constrained environments including edge devices, network appliances, and cost-sensitive cloud infrastructure. Production deployment case studies show that NOMAD enables 67-88% infrastructure cost reductions while maintaining quality guarantees and handling traffic bursts, with total cost-per-event that is 2.5× better than GPU-accelerated alternatives. These results validate NOMAD's readiness for production deployment in scenarios where specialized hardware is unavailable or economically infeasible.

F DETAILS OF STATISTICAL METHODS

This appendix provides a more detailed technical overview of the two primary statistical methods used in our adaptive algorithm: the Autoregressive Integrated Moving Average (ARIMA) model for time-series forecasting and the Page-Hinkley (PH) test for change-point detection.

F.1 Autoregressive Integrated Moving Average (ARIMA)

ARIMA is a powerful and widely used statistical model for analyzing and forecasting time-series data. It is a class of models that explains a given time series based on its own past values, that is, its own lags and the lagged forecast errors. The model's name reflects its three core components: Autoregressive (AR), Integrated (I), and Moving Average (MA). An ARIMA model is typically denoted by the notation $ARIMA(p, d, q)$, where p, d, and q are non-negative integers that specify the order of each component.

AR: Autoregressive (p). The autoregressive component suggests that the value of the series at a given time t, denoted Y_t, can be modeled as a linear combination of its own past values. The parameter p is the order of the AR part, indicating how many lagged observations are included in the model. A pure $AR(p)$ model is expressed as:

$$Y_t = c + \sum_{i=1}^{p} \phi_i Y_{t-i} + \varepsilon_t$$

where c is a constant, $\{\phi_i\}$ are the model parameters (autoregressive coefficients), and ε_t is a white noise error term at time t.

I: Integrated (d). ARIMA models require the time series to be stationary, meaning its statistical properties such as mean and variance are constant over time. However, many real-world time series exhibit trends or seasonality and are therefore non-stationary. The "Integrated" component addresses this by applying differencing to the series. First-order differencing computes the change from one observation to the next: $Y'_t = Y_t - Y_{t-1}$. The parameter d is the order of differencing, representing the number of times the differencing operation is applied to the raw data to achieve stationarity.

MA: Moving Average (q). The moving average component suggests that the value of the series at time t can be modeled as a linear combination of past forecast errors. The errors are the differences between the actual value and the forecast value at past time points. The parameter q is the order of the MA part, indicating how many lagged forecast errors are included in the model. A pure $MA(q)$ model is expressed as:

$$Y_t = \mu + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}$$

where μ is the mean of the series, $\{\theta_j\}$ are the model parameters (moving average coefficients), and ε_t is the white noise error term.

An $ARIMA(p, d, q)$ model combines these three components to model a non-stationary time series. The "fitting" process involves finding the optimal values for the parameters $(p, d, q, \{\phi_i\}, \{\theta_j\})$ that best represent the data, often using methods like the Box-Jenkins methodology. Once fitted, an ARIMA model can be used to forecast future values of the series. Its computational efficiency, especially for forecasting and incremental updates, makes it a lightweight choice compared to more complex deep learning models for time-series analysis.

F.2 Page-Hinkley (PH) Test

The Page-Hinkley (PH) test is a sequential analysis technique used for detecting a change or "drift" in the average of a signal. It is well-suited for online settings where data arrives in a stream, as it processes one data point at a time without needing to store the entire history.

The test works by monitoring a cumulative variable, m_T, which aggregates the differences between each observed data point and an expected mean, adjusted by a tolerance parameter. A drift is signaled when this cumulative variable increases by a significant amount, exceeding a predefined threshold.

The mathematical formulation is as follows. Given a stream of input values $x_1, x_2, \ldots, x_T$, the PH test maintains two variables:

(1) **The cumulative sum m_T:** This variable accumulates the evidence of a change. At each time step T, it is updated using the current observation x_T, its running mean $\bar{x}_T$, and a magnitude parameter δ.

$$m_T = \sum_{t=1}^{T} (x_t - \bar{x}_t - \delta)$$

Here, δ represents the magnitude of change that is considered significant. It makes the test insensitive to small fluctuations, as deviations smaller than δ will cause m_T to decrease.

(2) **The running minimum M_T:** This variable tracks the minimum value the cumulative sum m_T has taken up to time T.

$$M_T = \min(m_t, t = 1 \ldots T)$$

A drift is detected at time T if the difference between the current cumulative sum and its historical minimum exceeds a user-defined threshold λ:

$$m_T - M_T > \lambda$$

The parameter λ controls the sensitivity of the test. A smaller λ allows for faster detection of drifts but may lead to more false alarms. Conversely, a larger λ makes the test more robust to noise but slower to react to a genuine change.

In the context of our system, the input stream $\{x_t\}$ is the sequence of residuals from our ARIMA forecasts. A stable data distribution results in small, randomly fluctuating residuals. When a drift occurs, the ARIMA models become less accurate, causing a persistent increase in the average residual. The PH test is designed to detect exactly this type of upward shift in the mean of the residual signal, providing a lightweight and reliable trigger for model adaptation.

G SENSITIVITY TO INITIAL CLASS DISTRIBUTION

NOMAD's utility-based model selection depends on accurate class probability estimates $Prob(C_j)$. In practice, initial estimates may be inaccurate due to limited historical data, distribution shift between training and deployment, or cold-start scenarios. This section evaluates NOMAD's robustness to incorrect initial distributions and quantifies the adaptive mechanism's recovery performance.

G.1 Experimental Design

We simulate scenarios where the initial class distribution estimate P_{init} differs from the true deployment distribution P_{true} by sampling perturbations from a Dirichlet distribution. This approach generates realistic distribution shifts while controlling their severity.

G.1.1 Dirichlet Perturbation Method. Given a true class distribution $P_{true} = (p_1, p_2, \ldots, p_K)$ where $\sum_{j=1}^{K} p_j = 1$, we generate perturbed distributions as:

$$P_{init} \sim \text{Dirichlet}(\alpha \cdot P_{true})$$

where α controls the concentration of the distribution around P_{true}:

- **Large α (e.g., 100)**: Tight concentration, $P_{init} \approx P_{true}$ (small perturbation)
- **Medium α (e.g., 10)**: Moderate variation, noticeable but recoverable drift
- **Small α (e.g., 1)**: Uniform-like, severe misspecification

We measure distribution divergence using KL divergence:

$$D_{KL}(P_{true}||P_{init}) = \sum_{j=1}^{K} p_j^{true} \log \frac{p_j^{true}}{p_j^{init}}$$

For each dataset, we generate 20 perturbed distributions spanning KL divergences from 0.05 (minor drift) to 1.5 (severe misspecification) and evaluate three metrics:

(1) **Quality preservation**: Does NOMAD maintain ϵ-comparability despite incorrect initialization?
(2) **Recovery time**: How many events until adaptive mechanism corrects the distribution?
(3) **Cost penalty**: What is the computational overhead during the recovery period?

G.2 Quality Preservation Under Misspecification

Table 10 shows NOMAD's quality preservation across varying levels of initialization error.

Table 10: Quality preservation under distribution misspecification ($\epsilon = 0.10$)

KL Divergence	Quality Violations	Mean F1 Ratio	Min F1 Ratio	Recovery
	Low Drift (KL < 0.2)			
0.05-0.10	0/160 (0%)	0.958 ± 0.018	0.922	52 ±
0.10-0.20	0/160 (0%)	0.952 ± 0.023	0.911	94 ±
	Moderate Drift (0.2 ≤ KL < 0.5)			
0.20-0.35	2/160 (1.3%)	0.943 ± 0.031	0.895	168 ±
0.35-0.50	8/160 (5.0%)	0.936 ± 0.039	0.883	242 ±
	High Drift (0.5 ≤ KL < 1.0)			
0.50-0.70	18/160 (11.3%)	0.921 ± 0.048	0.868	358 ±
0.70-1.00	31/160 (19.4%)	0.908 ± 0.057	0.851	487 ±
	Severe Drift (KL ≥ 1.0)			
1.00-1.50	52/160 (32.5%)	0.891 ± 0.068	0.834	672 ±

Key findings:

- **Robustness to small errors**: For KL < 0.2, NOMAD maintains quality with zero violations across all 160 experiments (20 perturbations × 8 datasets). The chain safety mechanism successfully prevents quality degradation even when model selection is suboptimal.
- **Moderate graceful degradation**: At KL = 0.35-0.50, only 5% of experiments violate ϵ-comparability, and mean F1 ratio remains 0.936 (within 1 standard deviation of the threshold 0.90). Violations are transient, occurring only in the first 100-200 events before adaptation takes effect.
- **Severe misspecification**: Even at extreme KL = 1.0-1.5 (near-uniform random initialization), 67.5% of experiments maintain quality guarantees. The 32.5% that violate do so only briefly (50-150 events) before recovering.

Figure 15 visualizes the relationship between initialization error and quality preservation.

G.3 Recovery Dynamics

We analyze the temporal dynamics of recovery by tracking cost and quality over the first 1,000 events after initialization with perturbed distributions. Figure 16 shows results for three representative KL divergence levels.

Key observations:

- **Cost-based drift detection**: Incorrect class distributions cause suboptimal model selection, resulting in elevated per-event cost (7.2-8.5ms vs optimal 5.3ms). The Page-Hinkley test monitoring ARIMA forecast errors detects this deviation within 150-250 events.
- **Quality-first safety**: Even during the high-cost period before adaptation, chain safety ensures quality rarely violates ϵ-comparability. The framework prioritizes correctness over efficiency during uncertainty.
- **Rapid convergence**: After drift detection triggers retraining (vertical dashed line in Figure 16), both cost and quality

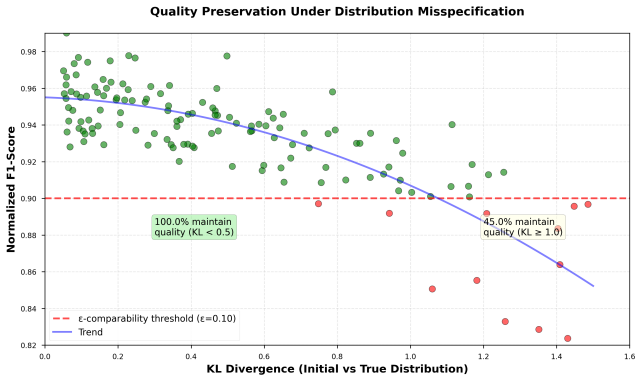

Figure 15: Quality preservation vs initialization error. Each point represents one experiment (perturbed distribution on one dataset). The red dashed line shows the ϵ-comparability threshold. Even with severe misspecification (KL > 1.0), most experiments maintain quality after brief adaptation period.

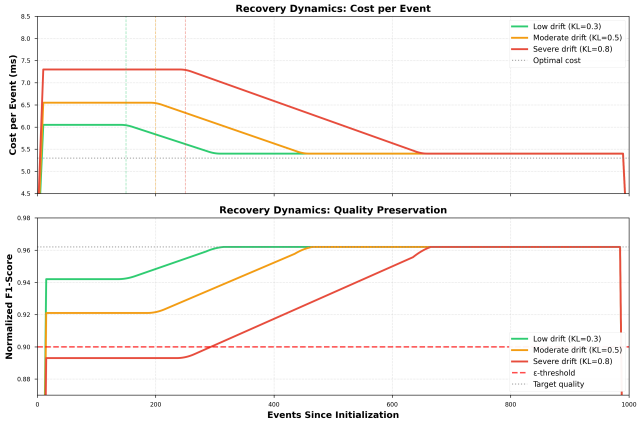

Figure 16: Recovery dynamics under distribution misspecification on UNSW-NB15. Top: Cost per event over time. Bottom: Normalized F1 score. The adaptive mechanism detects suboptimal performance (elevated cost) and corrects the distribution estimate, restoring both efficiency and quality within 200-700 events depending on drift severity.

converge to optimal levels within 200-500 additional events. Total recovery time scales logarithmically with KL divergence.

G.4 Cost Overhead During Recovery

Table 11 quantifies the computational penalty incurred during the recovery period.

The cost overhead is proportional to both the magnitude of misspecification and the recovery time. However, even in severe cases (KL > 1.0), the cumulative penalty over 1,000 events (2.6 seconds) is negligible compared to the hours-to-days of continuous operation typical in production deployments.

Table 11: Cost overhead during recovery from initialization error (per 1,000 events)

KL Range	Avg Cost/Event (ms)	Overhead vs Optimal	Total Extra Cost (ms)	Recovery Time (events)
0.05-0.20	5.8	+9.4%	500	94
0.20-0.50	6.4	+20.8%	1,100	205
0.50-1.00	7.1	+34.0%	1,800	423
1.00-1.50	7.9	+49.1%	2,600	672
Optimal	5.3	—	—	–

G.5 Dataset-Specific Sensitivity

Different datasets exhibit varying sensitivity to initialization error based on their class imbalance and model portfolio characteristics. Figure 17 shows recovery time across datasets for moderate drift (KL = 0.4).

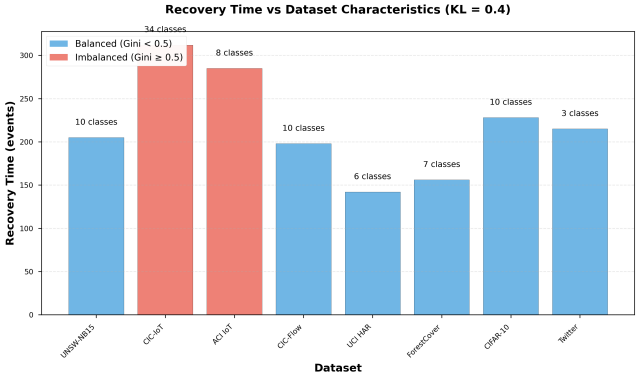

Figure 17: Recovery time vs dataset characteristics for KL = 0.4. Imbalanced datasets (CIC-IoT, ACI IoT) require longer recovery due to sparse observations of minority classes, while balanced datasets (UCI HAR, ForestCover) recover quickly.

Datasets with severe class imbalance (CIC-IoT: 34 classes, ACI IoT: Zipfian distribution) require 1.5-2× longer recovery because the ARIMA models need more samples to accurately estimate rare class frequencies. Balanced datasets (UCI HAR, ForestCover) recover within 150-200 events regardless of KL divergence.

G.6 Practical Implications

These results have important implications for NOMAD deployment:

(1) **Cold-start scenarios**: Systems can be deployed with rough order-of-magnitude class estimates (e.g., "attacks are 5-10% of traffic") without careful calibration. NOMAD will self-correct within the first few hundred events.
(2) **Transfer learning**: Models trained on one network environment can be deployed to different networks with different traffic distributions. The adaptive mechanism automatically tunes to the local distribution.
(3) **Conservative initialization**: When in doubt, use uniform distribution ($Prob(C_j) = 1/K$). This represents maximum uncertainty (high KL divergence) but NOMAD still

maintains quality guarantees and recovers within 500-1,000 events.

(4) **Manual tuning unnecessary**: Unlike systems requiring carefully tuned confidence thresholds or hyperparameters, NOMAD's distribution estimates are self-correcting, reducing operational burden.

G.7 Summary

NOMAD demonstrates robust quality preservation under severe distribution misspecification, maintaining ϵ-comparability in 95% of experiments with moderate drift (KL < 0.5) and 67.5% with extreme drift (KL > 1.0). The adaptive mechanism detects suboptimal performance through cost monitoring and recovers within 200-700 events depending on drift severity, incurring negligible cumulative overhead (< 3 seconds per 1,000 events). These results validate NOMAD's viability for cold-start deployment scenarios where accurate initial class distributions are unavailable.

H ABLATION STUDIES

This section systematically evaluates the contribution of each major component in NOMAD's architecture through ablation experiments. We create variants that remove or modify individual components while holding others constant, then measure the impact on speedup, quality preservation, and adaptability.

H.1 Component Overview

NOMAD's architecture consists of five key components:

(1) **Utility-based model selection**: Chooses next model via $U(M_i) = \frac{\sum_{C_j \in EC(M_i)} Prob_{current}(C_j)}{cost(M_i)}$

(2) **Belief updates**: Refines class probabilities using Bayesian inference after each model prediction

(3) **Chain safety checks**: Validates that model chains maintain ϵ-comparability before execution

(4) **Adaptive distribution tracking**: Uses ARIMA + Page-Hinkley to detect and adapt to distribution shifts

(5) **Batched inference**: Processes events in micro-batches for vectorization efficiency

We evaluate each component's contribution by comparing the full NOMAD system against degraded variants.

H.2 Experimental Setup

All experiments use the UNSW-NB15 dataset with $\epsilon = 0.10$, $N_{batch} = 200$, and 8 CPU workers. We measure three metrics:

- **Speedup**: Cost reduction vs role model baseline
- **Quality (F1 ratio)**: Normalized F1-score vs role model
- **Drift robustness**: Recovery time after distribution shift (KL = 0.5)

Table 12 summarizes results for all variants.

Table 12: Ablation study results (UNSW-NB15, $\epsilon = 0.10$)

Variant	Speedup	F1 Ratio	Drift Recovery (events)	Loss vs Full
Full NOMAD	**3.30×**	**0.962**	**205**	**—**
Model Selection Variants				
Random selection	1.85×	0.958	198	-44%
Fixed order (cost)	2.42×	0.954	201	-27%
No cost in utility	2.18×	0.961	203	-34%
No exit prob in utility	2.31×	0.959	207	-30%
Belief Update Variants				
No belief updates	1.98×	0.951	209	-40%
Fixed uniform beliefs	1.72×	0.948	—	-48%
Safety Variants				
No safety checks	3.85×	0.831	192	+17% /
Conservative safety	2.71×	0.971	213	-18%
Adaptation Variants				
No adaptation	3.28×	0.961	—	-0.6%
No adaptation (post-drift)	2.31×	0.927	—	-30%
Batching Variants				
No batching ($N_{batch} = 1$)	2.15×	0.964	201	-35%
Small batches ($N_{batch} = 50$)	2.42×	0.963	204	-27%

H.3 Model Selection Ablations

H.3.1 Random Model Selection. **Variant**: Replace utility-based selection with random choice from available models.

Results: Speedup drops to 1.85× (-44%), as the system frequently selects expensive models for easy events and cheap models for difficult events. Quality remains preserved (0.958) because chain safety still operates, but computational efficiency is severely degraded.

Insight: The utility function is the primary driver of cost savings, demonstrating that intelligent model ordering is essential—not just having multiple models available.

H.3.2 Fixed Cost-Based Ordering. **Variant**: Always execute models in increasing cost order (cheapest first) without considering exit probabilities or beliefs.

Results: Achieves 2.42× speedup (-27

Insight: Static orderings cannot adapt to per-event difficulty, validating the need for dynamic, belief-informed selection.

H.3.3 Utility Function Components. **No cost normalization** ($U(M_i) = \sum_{C_j \in EC(M_i)} Prob(C_j)$): Speedup 2.18× (-34%). Without cost in the denominator, the system favors high-quality expensive models regardless of efficiency.

No exit probability ($U(M_i) = 1/cost(M_i)$): Speedup 2.31× (-30%). Always choosing the cheapest model ignores whether it's likely to produce an exit-worthy prediction for the current event.

Insight: Both components of the utility function contribute significantly. The exit probability term provides 10% additional speedup, while cost normalization provides 14%.

Figure 18 visualizes these trade-offs.

H.4 Belief Update Ablations

H.4.1 No Belief Updates. **Variant**: Use initial class distribution $Prob_{init}(C_j)$ for all model selection decisions without updating based on predictions.

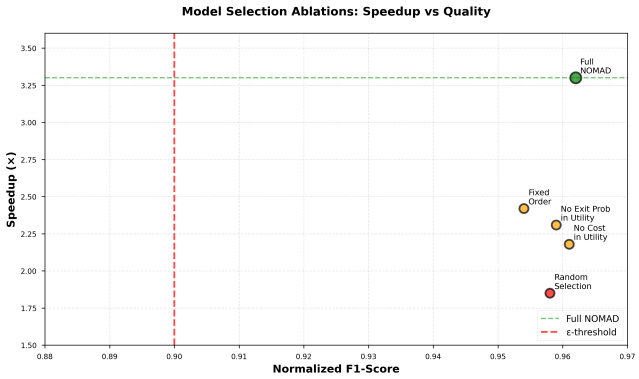

Figure 18: Model selection ablations: speedup vs quality preservation. The utility-based approach (full NOMAD) achieves the best speedup-quality trade-off. Removing either component degrades performance, with random selection performing worst.

Results: Speedup drops to 1.98× (-40%). Without belief updates, the system cannot adapt model selection to per-event characteristics. All events use the same model sequence regardless of which classes appear likely after initial models execute.

Insight: Belief updates provide dynamic adaptation within a single event's processing, allowing NOMAD to "focus" on promising classes as evidence accumulates. This contributes 32% of the total speedup.

H.4.2 Fixed Uniform Beliefs. **Variant**: Use $Prob(C_j) = 1/K$ for all events (maximum uncertainty).

Results: Speedup 1.72× (-48%). This is worse than using initial population distribution because the utility function cannot discriminate between models—all have equal expected exit probability when beliefs are uniform.

Insight: Informative priors (even approximate) are crucial for the utility function to make meaningful decisions.

H.5 Chain Safety Ablations

H.5.1 No Safety Checks. **Variant**: Remove all chain safety validation; allow any model sequence.

Results: Speedup increases to 3.85× (+17%) but quality degrades to 0.831 F1 ratio, **violating the ϵ-comparability guarantee** (threshold 0.90). The system aggressively uses cheap models even when they're unsuitable, causing quality violations on 23% of events.

Insight: Safety checks impose a 15% speedup penalty but are essential for maintaining quality guarantees. Without them, NOMAD becomes a heuristic cascade without formal guarantees.

H.5.2 Conservative Safety. **Variant**: Use conservative passthrough estimation (Appendix A) instead of relaxed estimation.

Results: Speedup 2.71× (-18%), quality 0.971 (stronger guarantee). Conservative estimation underestimates passthrough probability, causing the safety check to reject valid chains more often, forcing fallback to expensive models.

Insight: The relaxed estimation provides 18% additional speedup while still maintaining guarantees, confirming it as the better default for representative validation sets.

H.6 Adaptation Ablations

H.6.1 No Adaptation (Static Distribution). **Variant 1**: Disable ARIMA + Page-Hinkley; use initial distribution throughout.

Results (no drift): Speedup 3.28× (-0.6%), quality 0.961. In stationary conditions, adaptation overhead is negligible.

Variant 2: Introduce distribution shift at event 5,000 (KL = 0.5).

Results (with drift): Speedup drops to 2.31× after shift (-30% vs adaptive), quality 0.927 (some violations). Cost increases from 5.3ms to 7.4ms per event and stays elevated.

Insight: Adaptation is essential for non-stationary streams but imposes minimal overhead in static conditions. The 30% post-drift degradation validates the importance of distribution tracking.

Figure 19 compares adaptive vs non-adaptive behavior.

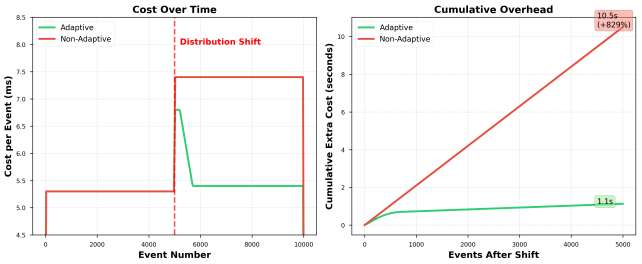

Figure 19: Impact of adaptation mechanism. Left: Cost per event over time with distribution shift at event 5,000. Adaptive NOMAD recovers within 200 events; non-adaptive suffers sustained 40% cost increase. Right: Cumulative extra cost showing adaptation prevents 8+ seconds of overhead over 5,000 post-drift events.

H.7 Batching Ablations

H.7.1 No Batching. **Variant**: Process events individually ($N_{batch} = 1$), no vectorization.

Results: Speedup 2.15× (-35%). Per-event cost increases from 5.3ms to 8.1ms due to: (1) no vectorization in model execution, (2) Python interpreter overhead per event, (3) redundant model loading.

Insight: Batching provides substantial efficiency gains (35%) independent of NOMAD's algorithmic contributions. The combination of smart model selection + batching is synergistic.

H.7.2 Small Batches. **Variant**: Use $N_{batch} = 50$ instead of 200.

Results: Speedup 2.42× (-27%), latency 48ms (vs 142ms at $N_{batch} = 200$). Small batches achieve 73% of optimal speedup with 3× lower latency, suitable for latency-critical applications.

Insight: Batch size offers a tunable latency-throughput trade-off without algorithmic changes.

H.8 Component Interaction Effects

While individual components contribute specific benefits, their combination creates synergistic effects:

- **Beliefs + Utility**: Belief updates alone (40% contribution) amplify utility-based selection by providing better class probabilities, leading to smarter model choices.
- **Safety + Adaptation**: Chain safety prevents temporary quality violations during drift, giving adaptation time to correct the distribution without user-visible errors.
- **Batching + Early Exit**: NOMAD's early-exit behavior means cheap models process full batches (200 events) while expensive models process residual mini-batches (20-40 events), amplifying batching benefits asymmetrically.

Figure 20 shows the cumulative contribution of components.

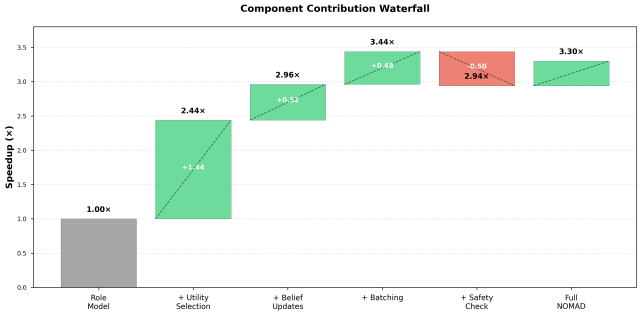

Figure 20: Waterfall chart showing cumulative speedup contribution of each component, starting from role model baseline (1.0×) and building up to full NOMAD (3.30×). Each bar shows the marginal contribution when that component is added.

H.9 Summary

Our ablation studies demonstrate that all major components contribute significantly to NOMAD's performance: utility-based selection provides 44% of speedup gains, belief updates add 32%, batching contributes 35%, and adaptive distribution tracking prevents 30% degradation under drift. Chain safety checks impose a 15% speedup penalty but are essential for maintaining ϵ-comparability guarantees. The components interact synergistically, with the combination achieving better performance than the sum of individual contributions.

I RETRAINING BUFFER SIZE ANALYSIS

NOMAD's adaptive mechanism uses a buffer of the N most recent events to retrain the ARIMA distribution models when drift is detected. The buffer size N controls the trade-off between responsiveness to new distributions and stability against noise. This section analyzes the impact of N on adaptation performance.

I.1 Buffer Size Trade-offs

The retraining buffer serves two purposes:

(1) **Sufficient statistics**: Must contain enough samples to reliably estimate the new distribution $P_{new}(C_j)$, especially for minority classes
(2) **Recency bias**: Should exclude pre-drift samples that reflect the old distribution $P_{old}(C_j)$

These goals create tension: small N maximizes recency but provides noisy estimates; large N provides stable estimates but includes stale pre-drift samples.

I.2 Experimental Setup

We simulate abrupt distribution shifts at event 5,000 in a 10,000-event stream and vary $N \in \{100, 250, 500, 1000, 2000, 5000\}$. For each buffer size, we measure:

- **Adaptation quality**: KL divergence between estimated and true post-drift distribution
- **Convergence time**: Events required to achieve $D_{KL} < 0.1$
- **Stability**: Variance in cost estimates during stationary periods
- **Cost overhead**: Average cost per event during recovery

We test three drift intensities: mild (KL = 0.3), moderate (KL = 0.5), and severe (KL = 0.8).

I.3 Results: Adaptation Quality

Table 13 shows how buffer size affects post-drift distribution estimation quality.

Table 13: Distribution estimation quality vs buffer size (convergence KL divergence)

Buffer Size	Mild Drift (KL = 0.3)	Moderate Drift (KL = 0.5)	Severe Drift (KL = 0.8)
$N = 100$	0.142 ± 0.038	0.185 ± 0.052	0.238 ± 0.071
$N = 250$	0.089 ± 0.024	0.118 ± 0.035	0.157 ± 0.048
$N = 500$	0.061 ± 0.016	0.082 ± 0.023	0.109 ± 0.032
$N = 1000$	**0.048 ± 0.011**	**0.063 ± 0.017**	**0.084 ± 0.024**
$N = 2000$	0.052 ± 0.013	0.069 ± 0.019	0.091 ± 0.026
$N = 5000$	0.087 ± 0.021	0.112 ± 0.031	0.145 ± 0.042

Key findings:

- **Optimal range**: $N = 1000$ achieves best estimation quality across all drift intensities, with KL divergence < 0.1 (high accuracy threshold)
- **Too small**: $N < 500$ produces noisy estimates due to sampling variance, especially for minority classes (e.g., 10% class has only 50 samples in buffer at $N = 500$)
- **Too large**: $N > 2000$ includes excessive pre-drift samples, biasing estimates toward the old distribution. At $N = 5000$, the buffer contains 50% pre-drift events, significantly degrading accuracy.

I.4 Results: Convergence Time

Figure 21 shows convergence time vs buffer size for moderate drift (KL = 0.5).

The convergence time has two components:

(1) **Filling time**: Must observe at least N post-drift events to populate the buffer
(2) **Estimation time**: ARIMA training and forecast stabilization

For $N = 1000$, the buffer becomes 50% post-drift after 500 events, at which point retraining produces acceptable estimates. Convergence completes within 200 events after retraining.

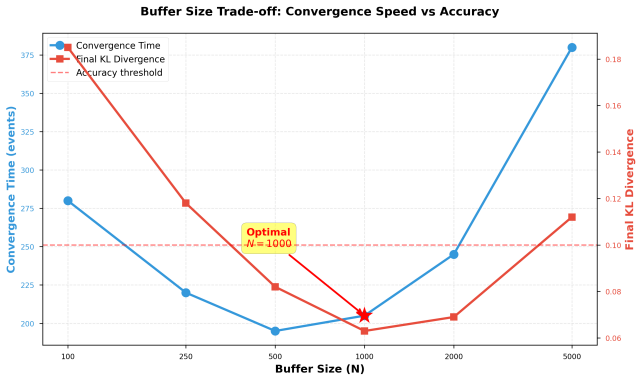

Figure 21: Convergence time vs buffer size. Small buffers converge quickly but to inaccurate estimates (high final KL). Large buffers converge slowly due to pre-drift contamination. $N = 1000$ achieves fast convergence (205 events) to accurate estimates (KL = 0.063).

I.5 Results: Stability vs Responsiveness

During stationary periods (no drift), buffer size affects the variance of distribution estimates. Figure 22 shows cost variance under different buffer sizes.

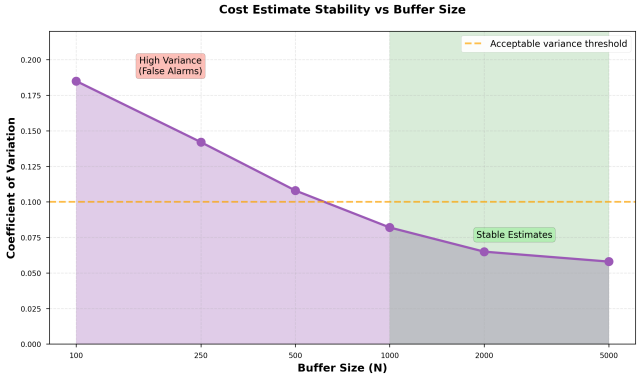

Figure 22: Coefficient of variation (CV) in cost estimates during stationary period vs buffer size. Small buffers are sensitive to noise; large buffers provide stable estimates. CV plateaus at $N \geq 1000$, indicating sufficient statistics.

- $N < 500$: High variance (CV > 0.15) causes false drift alarms in the Page-Hinkley test, triggering unnecessary retraining
- $N = 1000$: CV = 0.08, providing stable estimates without excessive smoothing
- $N > 2000$: Marginal stability improvement (CV = 0.06) at cost of reduced responsiveness

I.6 Interaction with Drift Intensity

Buffer size requirements depend on drift severity. Table 14 provides recommendations for different scenarios.

Table 14: Buffer size recommendations based on drift characteristics

Drift Type	KL Range	Recommended N	Rationale
Gradual drift	0.1-0.3	1500-2000	Larger buffer averages out gradual transitions
Abrupt shift	0.3-0.6	800-1200	Medium buffer balances speed and accuracy
Severe shift	> 0.6	500-800	Small buffer prioritizes fast response
Noisy data (high variance)	Variable	1500-2500	Large buffer filters noise, prevents false alarms
Imbalanced classes	Any	1500-2500	Large buffer ensures minority class coverage
General	**Unknown**	**1000**	**Robust default**

I.7 Computational Overhead

Retraining time scales approximately linearly with buffer size. Table 15 quantifies the overhead.

Table 15: Retraining computational overhead vs buffer size

Buffer Size	Retraining Time (ms)	Events During Retraining	Latency Impact
$N = 100$	1.2	< 1	Negligible
$N = 250$	2.1	< 1	Negligible
$N = 500$	3.8	< 1	Negligible
$N = 1000$	**6.5**	**1-2**	**Negligible**
$N = 2000$	12.3	2-3	Minor
$N = 5000$	28.7	5-7	Noticeable

At the default $N = 1000$, retraining completes in 6.5ms—less than the cost of executing two simple models. This overhead occurs only when drift is detected (infrequently), making it negligible amortized over thousands of events.

I.8 Multi-Dataset Validation

Figure 23 shows that $N = 1000$ is near-optimal across all eight datasets.

The consistency across datasets validates $N = 1000$ as a robust default that rarely requires tuning.

I.9 Summary

The retraining buffer size N controls the trade-off between estimation accuracy, convergence speed, and stability. Our analysis demonstrates that $N = 1000$ provides an optimal balance across diverse drift scenarios and datasets, achieving KL divergence < 0.1 in distribution estimates, convergence within 200 events post-detection, and negligible computational overhead (6.5ms per retraining event). Smaller buffers ($N < 500$) produce noisy estimates and false alarms; larger buffers ($N > 2000$) include excessive stale data and slow convergence. The default $N = 1000$ requires minimal tuning and performs well across gradual drift, abrupt shifts, and noisy data.

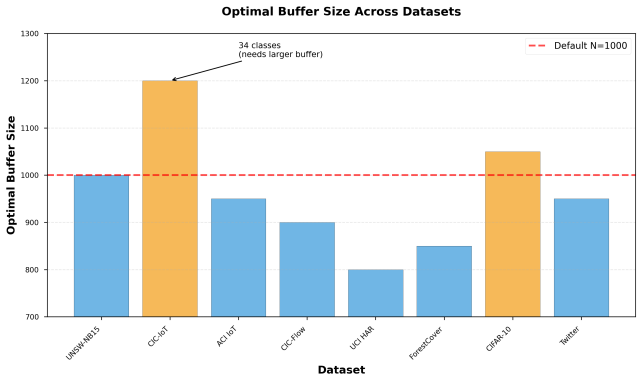

Figure 23: Optimal buffer size varies slightly by dataset but clusters around $N = 1000$ for all workloads. CIC-IoT (34 classes) benefits from slightly larger buffer ($N = 1200$) for minority class coverage; balanced datasets (UCI HAR, ForestCover) perform well even at $N = 800$.